ERC-7575 / WERC Vault — Detailed Remediation Report

Exhaustive, per-fix analysis with annotated before/after code, for all changes since commit 67aa1d7

2026-05-27

Contents

1 Detailed Remediation Report — all fixes since `67aa1d7`

Baseline: `67aa1d7` — *"fix: address audit findings (M-01, M-03, L-01, L-02, L-05, L-07, L-09, L-10, L-11, L-12)"* (2026-03-17). **Through:** `7e346f5` (2026-05-27), the final-review remediation commit. **Range:** 28 commits. **Test status:** `forge test` → **804 passed, 0 failed, 0 skipped** (100 suites). `forge build` clean. Final deep-review signoff: **no open Critical/High/Medium/Low contract-code findings**.

1.1 How to read this report

Each fix is documented with: **Severity**, **Problem** (the precise mechanism plus a concrete failure/attack scenario), **Root cause**, **Fix** (exactly what changed, before→after), **Files**, and **Tests** — followed by a **Code edited (before → after)** block showing the actual source that changed, taken verbatim from the commit. In those blocks, **red-tinted** snippets (marked `// BEFORE`) are the buggy/old code and **green-tinted** snippets (marked `// AFTER`) are the fixed code; Solidity is syntax-highlighted. Snippets

are trimmed to the relevant lines (with `// ...` marking elisions); pure dependency/gas/doc commits show the key changed line, docblock, or modifier.

> **Note on finding IDs.** IDs were reassigned per audit round, so the same label (e.g. "H-01", "M-01", "L-01") recurs across rounds with different meanings. Sections are therefore keyed by **commit hash** and described by mechanism, not by ID alone.

The work spans seven themes: (1) vault lifecycle & decommission safety, (2) investment integration & NAV, (3) ERC standards conformance, (4) WERC settlement & rBalance accounting, (5) async request/claim correctness, (6) upgrade & deployment hardening, (7) assurance & documentation. **Part A** covers the 25 committed audit-round fixes (`67aa1d7..cf912f1`); **Part B** covers the `f6b3fad` batch; **Part C** covers the final hardening + cleanup batch (`7e40f39` , `7e346f5`) and the final-review signoff.

---

2 Part A — Committed audit-round fixes (`67aa1d7..cf912f1`)

Presented in commit order (which follows the audit rounds).

2.0.1 Harden UUPS upgrades (proxiableUUID guard + implementation validation) and asset-exact withdraw rounding (`6f16122`)

- **Severity:** Hardening (with a Low-severity functional bug fixed alongside)
- **Problem:** Two distinct issues. (1) **Unvalidated UUPS upgrade target.** `upgradeTo` / `upgradeToAndCall` on both `ERC7575VaultUpgradeable` and `ShareTokenUpgradeable` forwarded the new implementation straight into `ERC1967Utils.upgradeToAndCall(newImplementation, "")` with no validation. The owner could point the proxy at an address that is not a valid UUPS implementation — an EOA, a non-proxiable contract, or another *proxy*. Installing such an implementation bricks the proxy permanently: because UUPS keeps the upgrade logic in the implementation, there is no further upgrade path to recover, freezing the proxy and all state behind it. (2) **Asset-exact withdraw could reject a valid partial claim.** `withdraw(assets, receiver, controller)` computed shares with `mulDiv(..., Floor)` ; for a partial claim flooring could yield `shares == 0` for a positive asset amount, and the close-out branch was keyed on the floored share count rather than the asset amount.
- **Root cause:** Upgrade entrypoints delegated to raw `ERC1967Utils.upgradeToAndCall` without the OZ `proxiableUUID` /rollback check, and exposed no `proxiableUUID()` to self-identify. Separately, withdraw used floor rounding on the share side of an asset-exact claim.
- **Fix:** (1) Added immutable `__self = address(this)` and a guarded `proxiableUUID()` returning `ERC1967Utils.IMPLEMENTATION_SLOT` (reverts `UUPSUnauthorizedCallContext()` when reached via delegatecall). `upgradeTo` / `upgradeToAndCall` route through a private `_upgradeToAndCallUUPS` that `try` s `IERC1822Proxiable(newImpl).proxiableUUID()` , reverts `UUPSUnsupportedProxiableUUID` if the slot mismatches, and `ERC1967InvalidImplementation` in the `catch` . (2) The withdraw close-out branch is selected on `assets == availableAssets` ; the partial branch flips to `Ceil` with guards `if (shares == 0) revert ZeroSharesCalculated()` and `if (shares >= availableShares) revert InsufficientClaimableShares()` . Also archived 6 unused contracts to `src/archive/` (excluded from build).
- **Files:** `ERC7575VaultUpgradeable.sol` , `ShareTokenUpgradeable.sol` (`proxiableUUID` ,

`_upgradeToAndCallUUPS`, `upgradeTo`, `upgradeToAndCall`, `withdraw`), `foundry.toml`.

- **Tests:** `AuditUpgradeFlow.t.sol` (`test_upgradeRejectsNonUUPSImpl`, `test_proxiableUUIDRevertsOnProxy`, `test_upgradePreservesState`, non-owner-cannot-upgrade, re-init guard).

Code edited (before → after)

`_claimWithdrawShares` — asset-exact withdraw rounding (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        uint256 availableAssets = $.claimableRedeemAssets[controller];
        if (assets > availableAssets) revert InsufficientClaimableAssets();

        // Calculate proportional shares for the requested assets
        uint256 availableShares = $.claimableRedeemShares[controller];
        shares = assets.mulDiv(availableShares, availableAssets, Math.Rounding.Floor);

        if (shares == availableShares) {
            // Remove from active redeem requesters if no more claimable assets and the potential dust
            $.activeRedeemRequesters.remove(controller);
            delete $.claimableRedeemAssets[controller];
            delete $.claimableRedeemShares[controller];
        } else {
            $.claimableRedeemAssets[controller] -= assets;
            $.claimableRedeemShares[controller] -= shares;
        }
```

```
// AFTER
        uint256 availableAssets = $.claimableRedeemAssets[controller];
        if (assets > availableAssets) revert InsufficientClaimableAssets();

        uint256 availableShares = $.claimableRedeemShares[controller];
        if (availableShares == 0) revert InsufficientClaimableShares();

        if (assets == availableAssets) {
            shares = availableShares;
            // Remove from active redeem requesters if no more claimable assets and the potential dust
            $.activeRedeemRequesters.remove(controller);
            delete $.claimableRedeemAssets[controller];
            delete $.claimableRedeemShares[controller];
        } else {
            // Asset-exact withdrawals must round shares up so a positive asset
            // claim cannot be satisfied by burning zero shares.
            shares = assets.mulDiv(availableShares, availableAssets, Math.Rounding.Ceil);
            if (shares == 0) revert ZeroSharesCalculated();
            if (shares >= availableShares) revert InsufficientClaimableShares();

            $.claimableRedeemAssets[controller] -= assets;
            $.claimableRedeemShares[controller] -= shares;
        }
```

`upgradeTo` / `_upgradeToAndCallUUPS` implementation guard (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function upgradeTo(address newImplementation) external onlyOwner {
        ERC1967Utils.upgradeToAndCall(newImplementation, "");
```

```
    }

    function upgradeToAndCall(address newImplementation, bytes calldata data) external payable onlyOwner
    {
        ERC1967Utils.upgradeToAndCall(newImplementation, data);
    }
```

```
// AFTER
    function upgradeTo(address newImplementation) external onlyOwner {
        _upgradeToAndCallUUPS(newImplementation, "");
    }

    function upgradeToAndCall(address newImplementation, bytes calldata data) external payable onlyOwner
    {
        _upgradeToAndCallUUPS(newImplementation, data);
    }

    function _upgradeToAndCallUUPS(address newImplementation, bytes memory data) private {
        try IERC1822Proxiable(newImplementation).proxiableUUID() returns (bytes32 slot) {
            if (slot != ERC1967Utils.IMPLEMENTATION_SLOT) {
                revert UUPSUnsupportedProxiableUUID(slot);
            }
        } catch {
            revert ERC1967Utils.ERC1967InvalidImplementation(newImplementation);
        }

        ERC1967Utils.upgradeToAndCall(newImplementation, data);
    }
```

`proxiableUUID` call-context guard — new (`ERC7575VaultUpgradeable.sol`)

```
// AFTER (new)
    address private immutable __self = address(this);

    error UUPSUnauthorizedCallContext();
    error UUPSUnsupportedProxiableUUID(bytes32 slot);

    /**
     * @dev ERC-1822 compatibility marker used to validate future upgrades.
     * Reverts when called through a proxy so a proxy cannot be accepted as a
     * future implementation and accidentally delegatecall into itself.
     */
    function proxiableUUID() external view returns (bytes32) {
        if (address(this) != __self) revert UUPSUnauthorizedCallContext();
        return ERC1967Utils.IMPLEMENTATION_SLOT;
    }
```

2.0.2 Upgrade OpenZeppelin 5.5.0 → 5.6.1 (`ee466e7`)

- **Severity:** Hardening (dependency bump)
- **Problem / Fix:** Mechanical. Bumped both OZ submodules to 5.6.1 and re-pointed the `Initializable` import to its canonical post-5.5.0 location (`@openzeppelin/contracts/proxy/utils/Initializ`
  No application logic changed; no storage-layout impact (ERC-7201 namespaced storage); suite re-run as the regression gate.

- **Files:** OZ submodules; import lines in `ERC7575VaultUpgradeable.sol`, `ShareTokenUpgradeable.sol`.

Code edited (before → after)

`Initializable` import re-point for OZ 5.6.1 (`ShareTokenUpgradeable.sol`)

```
// BEFORE
import {Initializable} from "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import {ERC20Upgradeable} from "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
import {IERC1822Proxiable} from "@openzeppelin/contracts/interfaces/draft-IERC1822.sol";
import {ERC1967Utils} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol";
```

```
// AFTER
import {ERC20Upgradeable} from "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
import {IERC1822Proxiable} from "@openzeppelin/contracts/interfaces/draft-IERC1822.sol";
import {ERC1967Utils} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol";
import {Initializable} from "@openzeppelin/contracts/proxy/utils/Initializable.sol";
```

2.0.3 Use ReentrancyGuardTransient (`86ccc2f`)

- **Severity:** Hardening (gas; reentrancy semantics unchanged)
- **Problem / Fix:** Not a vulnerability. Swapped `ReentrancyGuard` → `ReentrancyGuardTransient` on the vault, WERC vault, and WERC share token, and set `evm_version = "cancun"` to enable EIP-1153 `TSTORE` / `TLOAD`. The lock now lives in transient storage (auto-cleared per tx), adding no persistent slot (no layout impact) while preserving the same `nonReentrant` semantics. Berachain runs Prague ((superset of) Cancun), so the opcodes are available.
- **Files:** `ERC7575VaultUpgradeable.sol`, `WERC7575Vault.sol`, `WERC7575ShareToken.sol`, `foundry.toml`.

Code edited (before → after)

`ReentrancyGuard` → `ReentrancyGuardTransient` (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
// ...
contract ERC7575VaultUpgradeable is Initializable, ReentrancyGuard, Ownable2StepUpgradeable, IERC7540,
IERC7887, IERC165, IVaultMetrics, IERC7575Errors, IERC20Errors {
```

```
// AFTER
import {ReentrancyGuardTransient} from "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";
// ...
contract ERC7575VaultUpgradeable is Initializable, ReentrancyGuardTransient, Ownable2StepUpgradeable,
IERC7540, IERC7887, IERC165, IVaultMetrics, IERC7575Errors, IERC20Errors {
```

2.0.4 C-03: Scope ERC-7540 operators per-vault; restrict vaultTransferFrom (`a9471cf`)

- **Severity:** Critical
- **Problem:** Operator approvals were stored **centrally on the share token** via `setOperatorFor(controller, operator, approved)` gated only by `onlyVaults`. Any registered vault could therefore write `operators[victim][attacker] = true` for **any** victim,

and that grant was visible to **every** vault sharing the token. A single malicious/compromised vault could thus gain operator authority over any holder and drive their async requests across the whole multi-asset system. Compounding this, `vaultTransferFrom(from, to, amount)` (an `onlyVaults`, allowance-less escrow transfer) only rejected `to == address(0)`, so a rogue vault could pull a holder's shares to an **arbitrary** destination. Together: one rogue registered vault could self-authorize over any holder and redirect their escrowed shares — a cross-vault theft path. The trust boundary collapsed to "any one registered vault."

- **Root cause:** Operators modeled as a centralized table mutable by any vault, conflating per-Request authorization (which ERC-7540 scopes to the vault) with a global grant; `vaultTransferFrom` did not constrain the destination.
- **Fix:** (a) Operators moved to per-vault storage on the vault (`mapping(controller => mapping(operator => boo`; `setOperator` writes `$.operators[msg.sender][operator]`; all 10 request/claim auth sites read local `_isOperator`. The share token's `setOperator` / `isOperator` / `setOperatorFor`, the `operators` mapping, and `IERC7540Operator` inheritance were **removed**, eliminating the cross-vault grant path. (b) `vaultTransferFrom` now requires `to == msg.sender` (`ERC20InvalidReceiver` otherwise) — a vault can only escrow shares **into itself**.
- **Files:** `ERC7575VaultUpgradeable.sol` (operators storage, `setOperator`, `_isOperator`, 10 auth checks), `ShareTokenUpgradeable.sol` (removed central operator API; `vaultTransferFrom` restriction).
- **Tests:** `OperatorInterfaceCompliance.t.sol` (`test_OperatorsArePerVault`, `test_ShareToken_DoesNotImpl`; `EdgeCases_AuthorizationPaths.t.sol` rewritten to the per-vault API.

Code edited (before → after)

`setOperator` / `isOperator` — scope operators per-vault (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function setOperator(address operator, bool approved) public virtual returns (bool) {
        VaultStorage storage $ = _getVaultStorage();
        // Call setOperatorFor on the ShareToken to preserve the original msg.sender
        ShareTokenUpgradeable($.shareToken).setOperatorFor(msg.sender, operator, approved);
        // Emit event from vault level for compatibility with tests
        emit OperatorSet(msg.sender, operator, approved);
        return true;
    }

    function isOperator(address controller, address operator) external view returns (bool) {
        VaultStorage storage $ = _getVaultStorage();
        // Use a direct view call to the ShareToken's isOperator function
        return ShareTokenUpgradeable($.shareToken).isOperator(controller, operator);
    }
```

```
// AFTER
    function setOperator(address operator, bool approved) public virtual returns (bool) {
        if (operator == msg.sender) revert CannotSetSelfAsOperator();
        VaultStorage storage $ = _getVaultStorage();
        // Per-vault operator approval (ERC-7540): approval is scoped to this vault only.
        $.operators[msg.sender][operator] = approved;
        emit OperatorSet(msg.sender, operator, approved);
        return true;
    }
```

```
    function isOperator(address controller, address operator) external view returns (bool) {
        return _isOperator(controller, operator);
    }

    function _isOperator(address controller, address operator) internal view returns (bool) {
        return _getVaultStorage().operators[controller][operator];
    }
```

Every auth check now reads per-vault `_isOperator` (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        if (!(owner == msg.sender || IERC7540($.shareToken).isOperator(owner, msg.sender))) revert
        InvalidOwner();
```

```
// AFTER
        if (!(owner == msg.sender || _isOperator(owner, msg.sender))) revert InvalidOwner();
```

`vaultTransferFrom` — require `to == msg.sender` (`ShareTokenUpgradeable.sol`)

```
// BEFORE
        if (from == address(0)) {
            revert IERC20Errors.ERC20InvalidSender(address(0));
        }
        if (to == address(0)) {
            revert IERC20Errors.ERC20InvalidReceiver(address(0));
        }
```

```
// AFTER
        if (from == address(0)) {
            revert IERC20Errors.ERC20InvalidSender(address(0));
        }
        // Shares may only be pulled INTO the calling vault (its redemption escrow), never to an
        // arbitrary address. This bounds the allowance-less transfer power granted to registered
        // vaults: a vault can escrow a holder's shares for a redeem request, not redirect them.
        if (to != msg.sender) {
            revert IERC20Errors.ERC20InvalidReceiver(to);
        }
```

2.0.5 H-03: Loss-free vault decommission — `unregisterVault` can no longer orphan backing (`3c03bfc`)

- **Severity:** High
- **Problem:** A vault can be *quiescent* (deactivated, no pending/claimable/cancel obligations) yet still **hold assets**: claiming a deposit moves only the *shares* to the user; the backing assets stay in the vault and are excluded from the `reserved` accounting. The old `unregisterVault` validated only the request/cancel metrics and then removed the vault, deliberately not reverting on a nonzero raw balance. So an owner could unregister a quiescent-but-backed vault; the vault loses mint/burn rights and drops out of NAV while its backing remains stranded — total normalized backing falls while `totalSupply` is unchanged, **dropping NAV-per-share for every holder** (a socialized loss).

- **Root cause:** Unregistration never accounted for `totalAssets()` (claimed-deposit backing), and there was no path to remove a backed vault while preserving aggregate backing.
- **Fix:** Added `ERC7575VaultUpgradeable.migrateBackingOut(recipient)` — `nonReentrant`, `msg.sender == shareToken` only, re-validates quiescence directly from storage, then transfers `totalAssets()` out to `recipient` and returns the moved amount. On the share token, `unregisterVault` became a wrapper over a new internal `_migrateAndUnregisterVault(asset, address(0))`; a new `migrateAndUnregisterVault(fromAsset, toAsset)` (`onlyOwner`) uses the same core with a real `toAsset`. The core drains `fromVault`; **if `moved > 0`** it requires a non-zero `toAsset` (else `CannotUnregisterVaultAssetBalance` — so plain `unregisterVault` cannot orphan a backed vault), computes `amountIn = mulDiv(moved, fromScaling, toScaling, Ceil)` (ceil so the pool is never under-compensated), and `safeTransferFrom`s it into `toVault` (raising its `totalAssets()` with **no new shares minted**). `totalSupply` untouched, normalized NAV preserved.
- **Files:** `ERC7575VaultUpgradeable.sol` (`migrateBackingOut`), `ShareTokenUpgradeable.sol` (`unregisterVault`, `migrateAndUnregisterVault`, `_migrateAndUnregisterVault`, `VaultBackingMigrated`), `IERC7575Errors.sol` (`SameAssetMigration`).
- **Tests:** `MigrateAndUnregisterVault.t.sol` (9 cases incl. `test_migratePreservesBackingAndUnregisters`, `test_dustSweptAndCeilRounding`, `test_unregisterVaultRefusesBackedVault`).

Code edited (before → after)

`unregisterVault` — could orphan claimed-but-unredeemed backing (`ShareTokenUpgradeable.sol`)

```
// BEFORE
    function unregisterVault(address asset) external onlyOwner {
        if (asset == address(0)) revert ZeroAddress();
        ShareTokenStorage storage $ = _getShareTokenStorage();

        (bool exists, address vaultAddress) = $.assetToVault.tryGet(asset);
        if (!exists) revert AssetNotRegistered();

        try IVaultMetrics(vaultAddress).getVaultMetrics() returns (IVaultMetrics.VaultMetrics memory
        metrics) {
            if (metrics.isActive) revert CannotUnregisterActiveVault();
            if (metrics.totalPendingDepositAssets != 0) revert CannotUnregisterVaultPendingDeposits();
            if (metrics.totalClaimableRedeemAssets != 0) revert
            CannotUnregisterVaultClaimableRedemptions();
            if (metrics.totalCancelDepositAssets != 0) revert CannotUnregisterVaultAssetBalance();
            if (metrics.activeDepositRequestersCount != 0) revert
            CannotUnregisterVaultActiveDepositRequesters();
            if (metrics.activeRedeemRequestersCount != 0) revert
            CannotUnregisterVaultActiveRedeemRequesters();
        } catch {
            revert CannotUnregisterActiveVault();
        }
        // We do not revert on vault balance > 0 ... (assumed totalAssets excludes direct-transfer dust)
        $.assetToVault.remove(asset);
        delete $.vaultToAsset[vaultAddress];
        if ($.investmentShareToken != address(0)) {
            IERC20($.investmentShareToken).approve(vaultAddress, 0);
        }
        emit VaultUpdate(asset, address(0));
```

```
    }
```

```
// AFTER
    function unregisterVault(address asset) external onlyOwner {
        if (asset == address(0)) revert ZeroAddress();
        // Thin wrapper for removing an EMPTY vault: delegates to the shared core with a null toAsset.
        _migrateAndUnregisterVault(asset, address(0));
    }

    function migrateAndUnregisterVault(address fromAsset, address toAsset) external onlyOwner {
        _migrateAndUnregisterVault(fromAsset, toAsset);
    }

    function _migrateAndUnregisterVault(address fromAsset, address toAsset) internal {
        if (fromAsset == toAsset) revert SameAssetMigration();
        ShareTokenStorage storage $ = _getShareTokenStorage();
        (bool fromExists, address fromVault) = $.assetToVault.tryGet(fromAsset);
        if (!fromExists) revert AssetNotRegistered();

        // migrateBackingOut self-validates quiescence + inactive; moved == 0 => vault was empty.
        uint256 moved = ERC7575VaultUpgradeable(fromVault).migrateBackingOut(msg.sender);

        if (moved > 0) {
            if (toAsset == address(0)) revert CannotUnregisterVaultAssetBalance();
            (bool toExists, address toVault) = $.assetToVault.tryGet(toAsset);
            if (!toExists) revert AssetNotRegistered();

            // ceil so the pool is never under-compensated when scalingTo does not divide evenly.
            uint256 amountIn = Math.mulDiv(moved, ERC7575VaultUpgradeable(fromVault).getScalingFactor(),
            ERC7575VaultUpgradeable(toVault).getScalingFactor(), Math.Rounding.Ceil);

            if (amountIn > 0) {
                SafeTokenTransfers.safeTransferFrom(toAsset, msg.sender, toVault, amountIn);
            }
            emit VaultBackingMigrated(fromAsset, fromVault, moved, toAsset, toVault, amountIn);
        }

        $.assetToVault.remove(fromAsset);
        delete $.vaultToAsset[fromVault];
        if ($.investmentShareToken != address(0)) {
            IERC20($.investmentShareToken).approve(fromVault, 0);
        }
        emit VaultUpdate(fromAsset, address(0));
    }
```

`migrateBackingOut` — drain free backing on decommission — new (`ERC7575VaultUpgradeable.sol`)

```
// AFTER (new)
    function migrateBackingOut(address recipient) external nonReentrant returns (uint256 amount) {
        VaultStorage storage $ = _getVaultStorage();
        if (msg.sender != $.shareToken) revert Unauthorized();
        if ($.isActive) revert CannotUnregisterActiveVault();
        if ($.totalPendingDepositAssets != 0) revert CannotUnregisterVaultPendingDeposits();
        if ($.totalClaimableRedeemAssets != 0) revert CannotUnregisterVaultClaimableRedemptions();
        if ($.totalCancelDepositAssets != 0) revert CannotUnregisterVaultAssetBalance();
```

```
        if ($.activeDepositRequesters.length() != 0) revert
        CannotUnregisterVaultActiveDepositRequesters();
        if ($.activeRedeemRequesters.length() != 0) revert CannotUnregisterVaultActiveRedeemRequesters();

        amount = totalAssets();
        if (amount > 0) {
            SafeTokenTransfers.safeTransfer($.asset, recipient, amount);
        }
    }
```

2.0.6 H-04: Verify investment vault's `share()` matches the configured investment token (`243e85b`)

- **Severity:** High
- **Problem:** `_configureVaultInvestmentSettings` wired a vault to its investment target and granted an unlimited allowance **without checking the investment vault mints `investmentShareToken`**. If the investment vault's `share()` is some other token, assets invested through it mint a token `_calculateInvestmentAssets()` doesn't count: NAV **silently undercounts** invested capital — real assets leave but contribute nothing to NAV, dropping NAV-per-share for all holders. A governance misconfiguration produces a silent, ongoing accounting loss.
- **Root cause:** `registerVault` enforced `vault.share() == address(this)`, but the analogous investment-leg invariant was not checked.
- **Fix:** `_configureVaultInvestmentSettings` now guards `if (IERC7575(investmentVaultAddress).share() !=` before wiring/approving.
- **Files:** `ShareTokenUpgradeable.sol` (`_configureVaultInvestmentSettings`).
- **Tests:** `InvestmentVaultShareCheck.t.sol::test_setInvestmentShareToken_revertsOnShareMismatch`.

Code edited (before → after)

`_configureVaultInvestmentSettings` — verify investment vault `share()` (`ShareTokenUpgradeable.sol`)

```
// BEFORE
        // Configure investment vault if there's a matching one for this asset
        if (investmentVaultAddress != address(0)) {
            ERC7575VaultUpgradeable(vaultAddress).setInvestmentVault(IERC7575(investmentVaultAddress));

            // Grant unlimited allowance to the vault on the investment ShareToken
            IERC20(investmentShareToken).approve(vaultAddress, type(uint256).max);
        }
```

```
// AFTER
        if (investmentVaultAddress != address(0)) {
            // The investment vault must mint the configured investmentShareToken; otherwise invested
            // assets would land in a token _calculateInvestmentAssets does not count (NAV loss).
            if (IERC7575(investmentVaultAddress).share() != investmentShareToken) revert
            VaultShareMismatch();

            ERC7575VaultUpgradeable(vaultAddress).setInvestmentVault(IERC7575(investmentVaultAddress));
            IERC20(investmentShareToken).approve(vaultAddress, type(uint256).max);
        }
```

2.0.7 H-08: Document WERC7575Vault non-standard ERC-4626 redemption (`87b7e3a`)

- **Severity:** High (documentation of integration risk; no behavior change)
- **Problem:** `WERC7575Vault` presents an ERC-4626 surface but behaves non-standardly (par decimal-scaled conversion; `withdraw` / `redeem` require a validator-issued self-allowance even when `msg.sender == owner`; dual-gate when caller != owner; KYC-gated recipients) and had no contract-level documentation. Generic ERC-4626/ERC-20 integrators would hit opaque reverts.
- **Fix:** Added a `@title` / `@notice` / `@dev` header making the restricted behavior explicit and warning that standard integrations will fail. Documentation-only.
- **Files:** `WERC7575Vault.sol` (docblock).

Code edited (before → after)

`WERC7575Vault` — non-standard ERC-4626 redemption docblock — new (`WERC7575Vault.sol`)

```
// AFTER (doc)
/**
 * @title WERC7575Vault
 * @notice Synchronous ERC-7575 entry point for a single asset, paired with the restricted
 *         WERC7575ShareToken. Deposits/redeems convert at a fixed decimal-scaled par rate
 *         (1 asset <=> 1 normalized share); this is a wrapped receipt, not a yield-bearing ERC-4626.
 *
 * @dev NOT a standard ERC-4626 vault for integrators. Because the share token enforces restricted
 *      transfers, redemption deviates from ERC-4626:
 *      - withdraw()/redeem() require the `owner` to hold a validator-issued SELF-allowance on the
 *        share token (spendSelfAllowance) — even when msg.sender == owner. A plain owner redemption
 *        WILL revert without that permit.
 *      - when msg.sender != owner, BOTH the owner's self-allowance AND the caller's allowance are
 *        required (dual-gate).
 *      - all share recipients must be KYC-verified.
 *      Standard ERC-4626/ERC-20 integrations (DEXs, lenders, generic wallets) will fail.
 */
contract WERC7575Vault is IERC7575, ERC165, ReentrancyGuardTransient, Ownable2Step, Pausable,
IERC7575Errors {
```

2.0.8 M-01: WERC `unregisterVault` dust-DoS — port the migrate-and-compensate decommission (`ec2509a`)

- **Severity:** Medium
- **Problem:** `WERC7575ShareToken.unregisterVault` reverted whenever the vault's `totalAssets()` (its raw ERC-20 balance) was non-zero. An attacker could front-run a legitimate decommission by sending **1 wei** of the asset directly to the vault — `totalAssets()` becomes 1, the check trips `CannotUnregisterVaultAssetBalance`, and decommission is permanently grief-able (repeat after each sweep). There was also no loss-free way to remove a vault that legitimately still held backing.
- **Root cause:** The old comment wrongly assumed `totalAssets()` excludes direct-transfer dust; for this implementation `totalAssets()` *is* the raw balance, so any donation counts. No decommission primitive existed to drain the balance.
- **Fix:** Ported the H-03 pattern to the WERC stack: `WERC7575Vault.migrateBackingOut(recipient)` (share-token-only, requires deactivation) transfers out the **entire** balance (backing + dust) — so dust can no longer block removal; added `getScalingFactor()` for cross-decimal compensation; `unregisterVault` became a wrapper over `_migrateAndUnregisterVault(asset, address(0))`

(drain-and-compensate, ceil rounding favoring the pool), with `migrateAndUnregisterVault(from,to)` for loss-free removal of a backed vault.

- **Files:** `WERC7575ShareToken.sol`, `WERC7575Vault.sol`.
- **Tests:** `WERCMigrateAndUnregisterVault.t.sol` (incl. `test_dustSweptAndCeilRounding`, `test_unregisterVaultRefusesBackedVault`).

Code edited (before → after)

`unregisterVault` — reverted on any non-zero balance; falsely assumed `totalAssets` excluded dust (`WERC7575ShareToken.sol`)

```
// BEFORE
function unregisterVault(address asset) external onlyOwner {
    if (asset == address(0)) revert ZeroAddress();
    if (!_assetToVault.contains(asset)) revert AssetNotRegistered();

    address vaultAddress = _assetToVault.get(asset);

    // SAFETY CHECK: Validate that vault has no outstanding assets that users could claim
    try IERC7575Vault(vaultAddress).totalAssets() returns (uint256 totalAssets) {
        if (totalAssets != 0) revert CannotUnregisterVaultAssetBalance();
    } catch {
        revert("ShareToken: cannot verify vault has no outstanding assets");
    }
    // We do not revert on vault balance > 0 to prevent a DoS attack (M-01) ...
    // Checking `totalAssets` is sufficient as it excludes dust deposited directly via ERC20 transfer.

    _assetToVault.remove(asset);
    delete _vaultToAsset[vaultAddress];

    emit VaultUpdate(asset, address(0));
}
```

```
// AFTER
function unregisterVault(address asset) external onlyOwner {
    if (asset == address(0)) revert ZeroAddress();
    _migrateAndUnregisterVault(asset, address(0));
}

function _migrateAndUnregisterVault(address fromAsset, address toAsset) internal {
    if (fromAsset == toAsset) revert SameAssetMigration();
    (bool fromExists, address fromVault) = _assetToVault.tryGet(fromAsset);
    if (!fromExists) revert AssetNotRegistered();

    // migrateBackingOut self-validates deactivation; moved == 0 => vault was empty.
    uint256 moved = IERC7575Vault(fromVault).migrateBackingOut(msg.sender);

    if (moved > 0) {
        if (toAsset == address(0)) revert CannotUnregisterVaultAssetBalance();
        (bool toExists, address toVault) = _assetToVault.tryGet(toAsset);
        if (!toExists) revert AssetNotRegistered();

        // ceil so the pool is never under-compensated when scalingTo does not divide evenly.
        uint256 amountIn = Math.mulDiv(moved, IERC7575Vault(fromVault).getScalingFactor(),
        IERC7575Vault(toVault).getScalingFactor(), Math.Rounding.Ceil);
```

```
        if (amountIn > 0) {
            SafeTokenTransfers.safeTransferFrom(toAsset, msg.sender, toVault, amountIn);
        }
        emit VaultBackingMigrated(fromAsset, fromVault, moved, toAsset, toVault, amountIn);
    }

    _assetToVault.remove(fromAsset);
    delete _vaultToAsset[fromVault];
    emit VaultUpdate(fromAsset, address(0));
}
```

`migrateBackingOut` — drains the full balance (backing + dust) — new (`WERC7575Vault.sol`)

```
// AFTER (new)
function migrateBackingOut(address recipient) external nonReentrant returns (uint256 amount) {
    if (msg.sender != address(_shareToken)) revert Unauthorized();
    if (_isActive) revert CannotUnregisterActiveVault();

    amount = totalAssets();
    if (amount > 0) {
        SafeTokenTransfers.safeTransfer(_asset, recipient, amount);
    }
}
```

2.0.9 M-03: WERC `max*` ERC-4626 compliance — KYC, pause, vault liquidity (`b90e2f8`)

- **Severity:** Medium
- **Problem:** ERC-4626 requires `max*` never to exceed what the action accepts without reverting. (1) `maxDeposit` / `maxMint` returned `type(uint256).max` regardless of receiver, but minting is KYC-gated, so for a non-KYC receiver the executable max is 0. (2) `maxWithdraw` / `maxRedeem` returned the owner's **global** share balance, but WERC shares are fungible across all vaults while liquidity is per-vault — so e.g. `vaultB.maxWithdraw(alice)` could report 100e18 when vaultB is dry, and the withdrawal reverts.
- **Root cause:** Deposit limits ignored the KYC mint gate; withdraw/redeem limits ignored per-vault liquidity.
- **Fix (exact factors):** `maxDeposit/maxMint(receiver)` = `(active && !paused && isKycVerified(receiver)` `maxWithdraw(owner)` = `paused ? 0 : min(convertToAssets(balanceOf(owner)), totalAssets())` ; `maxRedeem(owner)` = `paused ? 0 : min(balanceOf(owner), convertToShares(totalAssets()))` . (Withdraw/redeem are deliberately not `isActive` -gated so holders can exit a deactivated vault.)
- **Files:** `WERC7575Vault.sol` (`maxDeposit/Mint/Withdraw/Redeem`).
- **Tests:** `WERCMaxFunctionsM03.t.sol` (20 tests across KYC/pause/liquidity/decimals/executability).

Code edited (before → after)

`maxDeposit` / `maxMint` — unbounded cap ignoring receiver KYC (`WERC7575Vault.sol`)

```
// BEFORE
function maxDeposit(address) public view returns (uint256) {
    return (_isActive && !paused()) ? type(uint256).max : 0;
```

```
}

function maxMint(address) public view returns (uint256) {
    return (_isActive && !paused()) ? type(uint256).max : 0;
}
```

```
// AFTER
function maxDeposit(address receiver) public view returns (uint256) {
    return (_isActive && !paused() && _shareToken.isKycVerified(receiver)) ? type(uint256).max : 0;
}

function maxMint(address receiver) public view returns (uint256) {
    return (_isActive && !paused() && _shareToken.isKycVerified(receiver)) ? type(uint256).max : 0;
}
```

`maxWithdraw` / `maxRedeem` — returned global share balance, uncapped by vault liquidity/pause (`WERC7575Vault.sol`)

```
// BEFORE
function maxWithdraw(address owner) public view returns (uint256) {
    return _convertToAssets(_shareToken.balanceOf(owner), Math.Rounding.Floor);
}

function maxRedeem(address owner) public view returns (uint256) {
    return _shareToken.balanceOf(owner);
}
```

```
// AFTER
function maxWithdraw(address owner) public view returns (uint256) {
    if (paused()) return 0;
    return Math.min(_convertToAssets(_shareToken.balanceOf(owner), Math.Rounding.Floor), totalAssets());
}

function maxRedeem(address owner) public view returns (uint256) {
    if (paused()) return 0;
    return Math.min(_shareToken.balanceOf(owner), _convertToShares(totalAssets(), Math.Rounding.Floor));
}
```

2.0.10 M-04: Scope ERC-7887 request blocking to the Pending cancelation state only (`1b4871a`)

- **Severity:** Medium
- **Problem:** EIP-7887 blocks new requests only "while cancelation is pending," distinguishing Pending from Claimable. The implementation tracked blocking via two `EnumerableSet` s added on cancel and removed only at **claim**, so the block spanned both Pending and Claimable. A controller who cancelled and had the cancelation fulfilled (→ Claimable) but not yet claimed could not submit a fresh request until claiming — a spec deviation that also disagreed with the contract's own `pendingCancel*Request()` views (which flip to `false` at fulfillment).
- **Root cause:** The block keyed on a set whose lifecycle (add-on-cancel / remove-on-claim) outlasted the "pending" predicate.
- **Fix:** Gate directly on the pending-cancelation balances (cleared at fulfillment): `requestDeposit`

reverts `DepositCancelationPending` iff `pendingCancelDepositAssets[controller] > 0`; `requestRedeem` reverts `RedeemCancelationPending` iff `pendingCancelRedeemShares[controller] > 0`. Deleted the two `EnumerableSet` fields and all their mutations.

- **Files:** `ERC7575VaultUpgradeable.sol` (`requestDeposit`, `requestRedeem`, cancel/claim functions, `VaultStorage`).
- **Tests:** `ERC7887Compliance.t.sol::test_{Deposit,Redeem}RequestUnblockedAfterFulfillBeforeClaim`.

Code edited (before → after)

`VaultStorage` — drop the two cancelation-tracking sets (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
// Total pending and claimable cancelation deposit assets (for totalAssets() calculation)
uint256 totalCancelDepositAssets;
// Track controllers with pending cancelations to block new requests
EnumerableSet.AddressSet controllersWithPendingDepositCancelations;
EnumerableSet.AddressSet controllersWithPendingRedeemCancelations;
```

```
// AFTER
// Total pending and claimable cancelation deposit assets (for totalAssets() calculation)
uint256 totalCancelDepositAssets;
```

`requestDeposit` — gate on the pending-cancel balance, not the set (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
// ERC7887: Block new deposit requests while cancelation is pending for this controller
if ($.controllersWithPendingDepositCancelations.contains(controller)) {
    revert DepositCancelationPending();
}
```

```
// AFTER
// ERC7887: block new deposit requests only while a cancelation is in the Pending state.
// pendingCancelDepositAssets is cleared at fulfillment (-> Claimable), so the gate lifts then,
// matching pendingCancelDepositRequest() and the spec's "while cancelation is pending" scope.
if ($.pendingCancelDepositAssets[controller] > 0) {
    revert DepositCancelationPending();
}
```

2.0.11 M-05: Align ERC-7887 cancelation events with the canonical spec ABI (`adf02a1`)

- **Severity:** Medium (interface/compliance)
- **Problem:** The cancelation events deviated from EIP-7887's signatures, so their `topic0` differed from what spec-aware indexers subscribe to — every cancelation transition was silently missed off-chain. The Request events carried extra params (indexed `owner`, amount); the Claim events were misnamed (`*Claimed`) and omitted `sender`.
- **Fix:** Updated `IERC7887.sol` and the four emit sites to the canonical signatures: `CancelDepositRequest(controller indexed, requestId indexed, sender)`, `CancelRedeemRequest(...)`, `CancelDepositClaim(controller indexed, receiver indexed, requestId indexed, sender, assets)`, `CancelRedeemClaim(...)`. Removed two dead events. Amounts remain queryable via the `pendingCancel*` / `claimableCancel*` views, so no information is lost.

- **Files:** `IERC7887.sol`, `ERC7575VaultUpgradeable.sol` (emit sites).
- **Tests:** `ERC7887Compliance.t.sol`, `OffChainHelperFunctions.t.sol` (updated `expectEmit` to canonical signatures).

Code edited (before → after)

`CancelDepositRequest` / `CancelDepositClaim` — non-canonical event ABI vs EIP-7887 (`interfaces/IERC7887.sol`)

```
// BEFORE
event CancelDepositRequest(address indexed controller, address indexed owner, uint256 indexed requestId,
address sender, uint256 assets);

event CancelDepositRequestClaimed(address indexed controller, address indexed receiver, uint256 indexed
requestId, uint256 assets);
```

```
// AFTER
event CancelDepositRequest(address indexed controller, uint256 indexed requestId, address sender);

event CancelDepositClaim(address indexed controller, address indexed receiver, uint256 indexed requestId,
address sender, uint256 assets);
```

emit sites — aligned to canonical signatures (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
emit CancelDepositRequest(controller, controller, REQUEST_ID, msg.sender, pendingAssets);
// ...
emit CancelDepositRequestClaimed(controller, receiver, REQUEST_ID, assets);
```

```
// AFTER
emit CancelDepositRequest(controller, REQUEST_ID, msg.sender);
// ...
emit CancelDepositClaim(controller, receiver, REQUEST_ID, msg.sender, assets);
```

2.0.12 M-06: Document that deactivated vaults correctly remain in NAV — invalidated finding (`54e6715`)

- **Severity:** Doc (false positive)
- **Why not a bug:** `isActive` is a deposit freeze, not a solvency signal. Deactivating a vault does not burn its already-minted shares (they keep circulating) nor remove its backing, so its assets *must* keep counting toward NAV; excluding them would drop real assets from the denominator while the shares circulate, instantly mispricing every holder downward. This is also what makes `migrateAndUnregisterVault` loss-free (backing leaves NAV only at unregister, never via the flag).
- **Fix:** Clarifying comments on `setVaultActive` and `getCirculatingSupplyAndAssets`. The legitimate depeg/par concern is tracked separately as C-01.
- **Files:** `ERC7575VaultUpgradeable.sol`, `ShareTokenUpgradeable.sol` (docblocks).

Code edited (before → after)

`setVaultActive` — clarify `isActive` is a deposit-freeze, not a NAV signal (`ERC7575VaultUpgradeable.sol`)

```
// AFTER (doc)
/**
 * @dev Sets the vault active state (only owner)
 *
 * NOTE: isActive is purely a DEPOSIT FREEZE — it gates requestDeposit/maxDeposit/maxMint and is a
 * precondition for migrateBackingOut. It is NOT a solvency/NAV signal: a deactivated vault still
 * holds backing for the shares it already minted (which remain in circulating supply), so its
 * assets MUST keep counting toward global NAV. Excluding them here would socialize a phantom loss
 * across all holders. To actually remove a vault's backing from NAV without loss, use
 * ShareTokenUpgradeable.migrateAndUnregisterVault (which preserves normalized NAV at par).
 */
```

2.0.13 M-07: Document the whole-token minimum-deposit model and its scope (`6587ab1`)

- **Severity:** Doc (accepted design)
- **Observation:** The minimum deposit is a whole-token threshold (`assets < minimumDepositAmount * 10^asset` `minimumDepositAmount` a `uint16`), not a raw-atomic or value-denominated minimum.
- **Why acceptable:** The deployment is stablecoin-only (~$1), so a whole-token floor is a sensible uniform dollar floor across decimals. The minimum is correctly vault-level (asset-denominated, uses the vault's `assetDecimals` , packed in a warm slot so the `requestDeposit` check costs no extra SLOAD / cross-contract call).
- **Fix:** Documented the granularity, the 65535-token cap, the vault-level rationale, and the explicit scope assumption (only suits ~par assets; a high-value/non-par asset would need a raw-atomic or oracle-denominated minimum). No behavior change.
- **Files:** `ERC7575VaultUpgradeable.sol` (`setMinimumDepositAmount` docblock).

Code edited (before → after)

`setMinimumDepositAmount` — document the whole-token model, vault-level rationale and scope (`ERC7575VaultUpgradeable.sol`)

```
// AFTER (doc)
/**
 * @dev Sets the minimum deposit amount (only owner), expressed as a WHOLE-TOKEN count.
 * @param _minimumDepositAmount Minimum deposit in whole tokens; the gate uses
 *        `_minimumDepositAmount * 10^assetDecimals`, so granularity is one whole token and the cap
 *        is 65535 tokens (no sub-token minima).
 *
 * WHY VAULT-LEVEL: the minimum is denominated in THIS vault's asset units and uses its
 * assetDecimals, so it belongs with the asset. It also keeps the requestDeposit hot path free —
 * minimumDepositAmount is packed in the same warm storage slot as asset/isActive/assetDecimals,
 * so the check needs no extra SLOAD and no cross-contract call.
 *
 * SCOPE ASSUMPTION: the whole-token model assumes ~par-valued assets (this deployment is
 * stablecoins — all ~$1). It is NOT value-aware: a high-value or non-par asset (e.g. 8-decimal
 * WBTC) would make the default an enormous floor and cannot express sub-token minima — that would
 * require switching to a raw-atomic (uint256) or oracle-denominated minimum.
 */
```

2.0.14 M-08: Reject zero controller/receiver at async request and claim boundaries (`cdcb1fe`)

- **Severity:** Medium

- **Problem:** Two zero-address fund-loss paths. (1) `requestDeposit` / `requestRedeem` accepted `controller == address(0)`: assets/shares are pulled into pending state keyed to `address(0)`, but no key can ever satisfy the claim auth (`controller == msg.sender` / operator), so the funds are permanently unclaimable. (2) `redeem` / `withdraw` / `claimCancel*` mutated state (burned shares, decremented claimables) **before** transferring to `receiver`, relying on the token to revert on transfer-to-zero; a permissive token would let state mutate and funds vanish.
- **Fix:** Added zero-address guards ahead of all state-dependent logic: `requestDeposit` / `requestRedeem` revert `ZeroAddress` on zero controller; `redeem` / `withdraw` / `claimCancelDepositRequest` revert `ERC20InvalidReceiver` on zero receiver; `claimCancelRedeemRequest` on zero owner.
- **Files:** `ERC7575VaultUpgradeable.sol` (6 functions).
- **Tests:** `ZeroAddressValidationM08.t.sol` (7 tests).

Code edited (before → after)

`requestDeposit` / `requestRedeem` — reject zero controller that would strand pulled funds (`ERC7575VaultUpgradeable.sol`)

```
// AFTER
function requestDeposit(uint256 assets, address controller, address owner) external nonReentrant returns
(uint256 requestId) {
    VaultStorage storage $ = _getVaultStorage();
    if (!$.isActive) revert VaultNotActive();
    if (controller == address(0)) revert ZeroAddress(); // a zero controller would strand the pulled
    assets (unclaimable)
    if (!(owner == msg.sender || _isOperator(owner, msg.sender))) revert InvalidOwner();
    // ...
}

function requestRedeem(uint256 shares, address controller, address owner) external nonReentrant returns
(uint256 requestId) {
    if (shares == 0) revert ZeroShares();
    if (controller == address(0)) revert ZeroAddress(); // a zero controller would strand the pulled
    shares (unclaimable)
    // ...
}
```

`redeem` / `claimCancelRedeemRequest` — reject zero receiver before CEI state changes (`ERC7575VaultUpgradeable.sol`)

```
// AFTER
function redeem(uint256 shares, address receiver, address controller) public nonReentrant returns
(uint256 assets) {
    VaultStorage storage $ = _getVaultStorage();
    if (receiver == address(0)) revert ERC20InvalidReceiver(receiver);
    if (!(controller == msg.sender || _isOperator(controller, msg.sender))) revert InvalidCaller();
    // ...
}

function claimCancelRedeemRequest(uint256 requestId, address owner, address controller) external
nonReentrant {
    if (requestId != REQUEST_ID) revert InvalidRequestId();
    VaultStorage storage $ = _getVaultStorage();
```

```
    if (owner == address(0)) revert ERC20InvalidReceiver(owner);
    if (!(controller == msg.sender || _isOperator(controller, msg.sender))) revert InvalidCaller();
    // ...
}
```

2.0.15 M-09: Fix ERC-165 discovery — drop `0x00000000` umbrellas, make `IERC7575Share` canonical (`af62748`)

- **Severity:** Medium (interface/compliance)
- **Problem:** (a) The vault advertised `type(IERC7540).interfaceId` and `type(IERC7887).interfaceId` — function-less *umbrella* interfaces whose ID is `0x00000000`, so `supportsInterface(0x00000000)` returned `true` (an ERC-165 hygiene violation). (b) The local `IERC7575Share` bundled `vault(address)` **and** `getRegisteredAssets()`, so its computed ID was `0x3749710f` instead of the canonical `0xf815c03d` (`vault(address)` alone), and `WERC7575ShareToken` advertised the wrong value — canonical ERC-7575 detection of the share token failed.
- **Fix:** (a) Removed the umbrella IDs (the canonical *sub*-interfaces are still advertised individually). (b) Moved `getRegisteredAssets()` into `IERC7575ShareExtended`, leaving `IERC7575Share` as exactly `vault(address)` → canonical `0xf815c03d`. Interface IDs before→after: share `0x3749710f → 0xf815c03d`; vault no longer returns `true` for `0x00000000`.
- **Files:** `ERC7575VaultUpgradeable.sol`, `WERC7575ShareToken.sol` (`supportsInterface`), `IERC7575.sol`.
- **Tests:** `ComprehensiveERC7540ERC7575Test.t.sol`, `WERC7575ShareTokenCoverageTests.t.sol`, `analysis/InterfaceIDCheck.t.sol` (canonical ID asserted, `0x00000000` asserted false).

Code edited (before → after)

`supportsInterface` — advertised `0x00000000` umbrella IDs (IERC7540 / IERC7887) (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
function supportsInterface(bytes4 interfaceId) public pure virtual returns (bool) {
    return interfaceId == type(IERC7575).interfaceId || interfaceId == type(IERC7540).interfaceId ||
    interfaceId == type(IERC7540Deposit).interfaceId
        || interfaceId == type(IERC7540Redeem).interfaceId || interfaceId ==
type(IERC7540Operator).interfaceId || interfaceId == type(IERC7887).interfaceId
        || interfaceId == type(IERC7887DepositCancelation).interfaceId || interfaceId ==
type(IERC7887RedeemCancelation).interfaceId || interfaceId == type(IERC165).interfaceId;
}
```

```
// AFTER
function supportsInterface(bytes4 interfaceId) public pure virtual returns (bool) {
    // Only the canonical sub-interface IDs are advertised. The IERC7540/IERC7887 umbrella interfaces
    // declare no functions, so type(...).interfaceId == 0x00000000 for them; advertising those would
    // make supportsInterface(0x00000000) return true (ERC-165 hygiene violation).
    return interfaceId == type(IERC7575).interfaceId || interfaceId == type(IERC7540Deposit).interfaceId
    || interfaceId == type(IERC7540Redeem).interfaceId
        || interfaceId == type(IERC7540Operator).interfaceId || interfaceId ==
type(IERC7887DepositCancelation).interfaceId || interfaceId ==
type(IERC7887RedeemCancelation).interfaceId
        || interfaceId == type(IERC165).interfaceId;
```

```
}
```

`IERC7575Share` — non-canonical interfaceId from a bundled `getRegisteredAssets` (`interfaces/IERC7575.sol`)

```
// BEFORE
interface IERC7575Share {
    function vault(address asset) external view returns (address vault);

    /// @dev Returns all registered assets in the multi-asset system.
    function getRegisteredAssets() external view returns (address[] memory assets);  // ID becomes
    0x3749710f
}
```

```
// AFTER
interface IERC7575Share {
    function vault(address asset) external view returns (address vault);            // canonical
    0xf815c03d
}

interface IERC7575ShareExtended is IERC7575Share {
    /// @dev Returns all registered assets in the multi-asset system.
    function getRegisteredAssets() external view returns (address[] memory assets);
}
```

2.0.16 C-01: Document the depeg-response runbook on `setVaultActive` (`015d9c1`)

- **Severity:** Doc (accepted residual)
- **Problem:** Cross-asset NAV is par with no on-chain oracle. If a backing asset depegs, NAV doesn't reprice; an arbitrageur could deposit the depegged asset at par and redeem against a healthy vault. An operator who only calls `setVaultActive(false)` is not protected — deactivation freezes the deposit leg but does not reprice, so redemptions keep overpaying at par.
- **Fix:** Documented (comment-only) the par assumption, that protection is the operator gate (fulfillment is `onlyInvestmentManager`; WERC adds validator self-allowance + KYC), and the 3-step runbook: (1) deactivate the depegged vault; (2) **withhold redeem fulfillment across all vaults**; (3) `migrateAndUnregisterVault` to swap the bad backing out (or socialize the loss).
- **Files:** `ERC7575VaultUpgradeable.sol` (`setVaultActive` docblock).

Code edited (before → after)

`setVaultActive` — depeg response runbook NatSpec (`ERC7575VaultUpgradeable.sol`)

```
// AFTER (doc)
 * DEPEG RESPONSE (C-01, accepted residual — no on-chain oracle): cross-asset NAV assumes par
 * (all assets ~$1). There is NO automatic repricing if an asset depegs. Protection is the
 * operator gate, not an oracle: every value-moving path is permissioned — deposit/redeem
 * fulfillment is onlyInvestmentManager (async), and the WERC stack additionally requires
 * validator self-allowance + KYC. Deactivating here only stops the DEPOSIT leg of a depeg
 * arbitrage; it does NOT reprice, so redemptions against healthy vaults would still overpay at
 * par. Full operator runbook on a depeg: (1) setVaultActive(false) on the depegged vault;
 * (2) WITHHOLD redeem fulfillment across vaults until repriced; (3) migrateAndUnregisterVault to
```

```
 * swap the bad backing out (or deliberately socialize the loss). See docs/DESIGN_ASSUMPTIONS.md.
```

2.0.17 Perf: remove self-external NAV call and `mulDiv`-by-1 on hot paths (`a09e49b`)

- **Severity:** Perf (no behavior change)
- **Fix:** Extracted internal `_getCirculatingSupplyAndAssets()` (the `convert*` functions now call it directly instead of `this.getCirculatingSupplyAndAssets()` — removing a `STATICCALL`-to-self per conversion); replaced `mulDiv(x, scalingFactor, 1)` with `x * scalingFactor` at the clean normalization sites (identical overflow semantics).
- **Files:** `ShareTokenUpgradeable.sol`, `ERC7575VaultUpgradeable.sol`.

Code edited (before → after)

`getCirculatingSupplyAndAssets` — remove the self external `STATICCALL` (`ShareTokenUpgradeable.sol`)

```
// BEFORE
function getCirculatingSupplyAndAssets() external view returns (uint256 circulatingSupply, uint256
totalNormalizedAssets) {
    ShareTokenStorage storage $ = _getShareTokenStorage();
    // ... aggregation loop ...
}
// elsewhere, on the hot conversion path:
    (uint256 circulatingSupply, uint256 totalNormalizedAssets) = this.getCirculatingSupplyAndAssets();
```

```
// AFTER
function getCirculatingSupplyAndAssets() external view returns (uint256 circulatingSupply, uint256
totalNormalizedAssets) {
    return _getCirculatingSupplyAndAssets();
}

/// @dev Internal core. Called directly by convert* to avoid an external STATICCALL to self.
function _getCirculatingSupplyAndAssets() internal view returns (uint256 circulatingSupply, uint256
totalNormalizedAssets) {
    ShareTokenStorage storage $ = _getShareTokenStorage();
    // ... aggregation loop ...
}
// hot path now calls the internal core directly:
    (uint256 circulatingSupply, uint256 totalNormalizedAssets) = _getCirculatingSupplyAndAssets();
```

`_convertToShares` — `mulDiv(x, scaling, 1)` replaced with a plain multiply (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        uint256 normalizedAssets = Math.mulDiv(assets, $.scalingFactor, 1);
```

```
// AFTER
        // mulDiv(x, scaling, 1) == x * scaling exactly; same overflow revert under checked arithmetic,
        cheaper.
        uint256 normalizedAssets = assets * $.scalingFactor;
```

2.0.18 Harden UUPS guard (`onlyProxy`) + document the investment-NAV `rBalanceOf` catch (`98bb526`)

- **Severity:** Low (defense-in-depth) + Doc
- **Fix:** Added `if (address(this) == __self) revert UUPSUnauthorizedCallContext();` at the top of `_upgradeToAndCallUUPS` in both upgradeable contracts (blocks running an upgrade against the raw implementation; not independently exploitable since `onlyOwner` already fires on the uninitialized impl, but matches OZ defense-in-depth). Documented the deliberate availability-over-precision tradeoff of the `try/catch` around `rBalanceOf` in `_calculateInvestmentAssets`.
- **Files:** `ERC7575VaultUpgradeable.sol`, `ShareTokenUpgradeable.sol`.

Code edited (before → after)

`_upgradeToAndCallUUPS` — `onlyProxy` context guard (`ERC7575VaultUpgradeable.sol`)

```
// AFTER
function _upgradeToAndCallUUPS(address newImplementation, bytes memory data) private {
    // onlyProxy: upgrades must run in proxy (delegatecall) context, never on the raw implementation.
    // Defense-in-depth — onlyOwner already blocks direct-impl calls (the impl is never initialized) —
    // but this matches the OZ UUPS guard explicitly.
    if (address(this) == __self) revert UUPSUnauthorizedCallContext();

    try IERC1822Proxiable(newImplementation).proxiableUUID() returns (bytes32 slot) {
    // ...
}
```

`_calculateInvestmentAssets` — document the `rBalanceOf` catch tradeoff (`ShareTokenUpgradeable.sol`)

```
// AFTER (doc)
        // Add rBalanceOf (reserved balance) if the investment share token supports it.
        // The catch is a deliberate availability-over-precision tradeoff: it lets a plain ERC-20
        // investment token (no rBalanceOf) be used via graceful degradation. Failing loud instead
        // would brick every conversion across all vaults (this getter feeds convertToShares/Assets).
        // balanceOf is intentionally NOT wrapped — a broken core balance should revert.
        try IWERC7575ShareToken(investmentShareToken).rBalanceOf(address(this)) returns (uint256 rShares)
        {
            totalInvestmentAssets += rShares;
        } catch {
            // rBalanceOf unsupported or reverted — continue with regular balance only (see note above).
        }
```

2.0.19 Support ERC-1271 signatures on WERC permit (`602a1bc`)

- **Severity:** Medium (feature/interop)
- **Problem:** WERC `permit` used raw `ECDSA.recover`, so contract accounts could not authorize approvals. WERC redemption depends on a validator-signed **self-allowance** (`permit` with `owner == spender`); once the validator becomes a multisig/contract (the planned migration), it could never produce an ECDSA signature — the redemption gate would be permanently unsatisfiable.
- **Fix:** Added a `permit(..., bytes signature)` overload verified via `SignatureChecker.isValidSignatureNow(` (accepts ECDSA **and** ERC-1271). The self-permit mechanism is preserved exactly: `expectedSigner = owner == spender ? _validator : owner` — for the self-allowance path

the required signer is the validator (now allowed to be a contract). The legacy 7-arg `(v,r,s)` overload became a thin wrapper, so EIP-2612 tooling is unchanged.

- **Files:** `WERC7575ShareToken.sol` (`permit` overloads; `ECDSA` → `SignatureChecker`).
- **Tests:** `PermitERC1271.t.sol` (contract validator self-allowance, contract owner approve, invalid inner sig, EOA regression).

Code edited (before → after)

`permit` — raw `ECDSA.recover` replaced with ERC-1271-aware `SignatureChecker` (`WERC7575ShareToken.sol`)

```
// BEFORE
    function permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32 r,
    bytes32 s) public virtual {
        if (block.timestamp > deadline) revert ERC2612ExpiredSignature(deadline);

        uint256 nonce = _useNonce(owner);
        // ... assembly struct-hash build ...
        bytes32 hash = _hashTypedDataV4(structHash);
        address signer = ECDSA.recover(hash, v, r, s);   // EOA-only: a contract validator can't sign

        if (owner == spender) {
            if (signer != _validator) revert ERC2612InvalidSigner(signer, owner);
        } else {
            if (signer != owner) revert ERC2612InvalidSigner(signer, owner);
        }
        _approve(owner, spender, value);
    }
```

```
// AFTER
    function permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32 r,
    bytes32 s) public virtual {
        permit(owner, spender, value, deadline, abi.encodePacked(r, s, v));  // legacy 7-arg now a thin
wrapper
    }

    function permit(address owner, address spender, uint256 value, uint256 deadline, bytes memory
    signature) public virtual {
        if (block.timestamp > deadline) revert ERC2612ExpiredSignature(deadline);

        uint256 nonce = _useNonce(owner);
        bytes32 hash = _hashTypedDataV4(keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value,
        nonce, deadline)));

        // Self-allowance must be authorized by the validator; all other approvals by the owner.
        address expectedSigner = owner == spender ? _validator : owner;
        if (!SignatureChecker.isValidSignatureNow(expectedSigner, hash, signature)) {  // accepts ECDSA
        AND ERC-1271
            revert ERC2612InvalidSigner(expectedSigner, owner);
        }
        _approve(owner, spender, value);
    }
```

2.0.20 Gate vault decommission on outstanding redeem cancelations (`5e0f1e1`)

- **Severity:** High (decommission desync)
- **Problem:** `migrateBackingOut` gated on every async obligation **except** redeem cancelations. A fulfilled-but-unclaimed `cancelRedeem` escrows shares owed back to the redeemer, but `cancelRedeemRequest` removed the controller from `activeRedeemRequesters` and there was no aggregate counter — so the vault could be unregistered while it still owed escrowed shares (only recoverable by an accident of the claim path using a plain `safeTransfer` rather than an `onlyVaults` mint/burn).
- **Fix:** Added `totalCancelRedeemShares` to `VaultStorage` (incremented in `cancelRedeemRequest`, decremented in `claimCancelRedeemRequest`) and the gate `if ($.totalCancelRedeemShares != 0) revert Ca`
- **Files:** `ERC7575VaultUpgradeable.sol` , `IERC7575Errors.sol` .
- **Tests:** `MigrateAndUnregisterVault.t.sol::test_revertIfRedeemCancelationOutstanding` .

Code edited (before → after)

`migrateBackingOut` — gate on outstanding redeem cancelations (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        if ($.totalPendingDepositAssets != 0) revert CannotUnregisterVaultPendingDeposits();
        if ($.totalClaimableRedeemAssets != 0) revert CannotUnregisterVaultClaimableRedemptions();
        if ($.totalCancelDepositAssets != 0) revert CannotUnregisterVaultAssetBalance();
        if ($.activeDepositRequesters.length() != 0) revert
        CannotUnregisterVaultActiveDepositRequesters();
```

```
// AFTER
        if ($.totalPendingDepositAssets != 0) revert CannotUnregisterVaultPendingDeposits();
        if ($.totalClaimableRedeemAssets != 0) revert CannotUnregisterVaultClaimableRedemptions();
        if ($.totalCancelDepositAssets != 0) revert CannotUnregisterVaultAssetBalance();
        if ($.totalCancelRedeemShares != 0) revert CannotUnregisterVaultRedeemCancelationsPending();
        if ($.activeDepositRequesters.length() != 0) revert
        CannotUnregisterVaultActiveDepositRequesters();
```

`cancelRedeemRequest` / `claimCancelRedeemRequest` — maintain the new aggregate counter (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        // Move from pending to pending cancelation
        delete $.pendingRedeemShares[controller];
        $.pendingCancelRedeemShares[controller] = pendingShares;
        // ... and at claim:
        delete $.claimableCancelRedeemShares[controller];
        SafeTokenTransfers.safeTransfer($.shareToken, owner, shares);
```

```
// AFTER
        // Move from pending to pending cancelation
        delete $.pendingRedeemShares[controller];
        $.pendingCancelRedeemShares[controller] = pendingShares;
        $.totalCancelRedeemShares += pendingShares; // tracks the escrowed-shares obligation through
claim
        // ... and at claim:
```

```
        delete $.claimableCancelRedeemShares[controller];
        $.totalCancelRedeemShares -= shares;
        SafeTokenTransfers.safeTransfer($.shareToken, owner, shares);
```

2.0.21 M-01: Enforce investment `share()` guard on the direct `setInvestmentVault` path (`4474f87`)

- **Severity:** Medium
- **Problem:** The centralized config path validated `investmentVault.share() == investmentShareToken` (H-04), but `setInvestmentVault` is owner-callable directly and only checked `asset()`. A direct call with a vault whose `asset()` matches but `share()` mints an uncounted token causes subsequent `investAssets` to move idle assets into shares NAV never sees — silent under-collateralization.
- **Fix:** `setInvestmentVault` now also fetches the configured `investmentShareToken` and reverts `VaultShareMismatch` if it is unset or `investmentVault_.share() != configuredInvestmentShareToken`.
- **Files:** `ERC7575VaultUpgradeable.sol` (`setInvestmentVault`).
- **Tests:** `InvestmentVaultShareCheck.t.sol::test_setInvestmentVault_directCall_revertsOnShareMismatch`

Code edited (before → after)

`setInvestmentVault` — enforce `investmentShareToken` match on the direct path (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        if (address(investmentVault_.asset()) != $.asset) revert AssetMismatch();
        $.investmentVault = address(investmentVault_);
        emit InvestmentVaultSet(address(investmentVault_));
```

```
// AFTER
        if (address(investmentVault_.asset()) != $.asset) revert AssetMismatch();
        address configuredInvestmentShareToken =
        ShareTokenUpgradeable($.shareToken).getInvestmentShareToken();
        if (configuredInvestmentShareToken == address(0) || investmentVault_.share() !=
        configuredInvestmentShareToken) {
            revert VaultShareMismatch();
        }
        $.investmentVault = address(investmentVault_);
        emit InvestmentVaultSet(address(investmentVault_));
```

2.0.22 M-02/03/04: Reject zero-address entries in settlement batch transfers (`b9d27ea`)

- **Severity:** Medium
- **Problem:** `batchTransfers` / `rBatchTransfers` bypass the ERC-20 zero-address guard. A creditor of `address(0)` shuffles real balance into `_balances[address(0)]` with no `totalSupply` change — permanent value loss and a `Σbalances != totalSupply` drift. (Validator-only, so operator/engine-bug class, not attacker-reachable — but cheap to harden on a regulated path.)
- **Fix:** Added `if (debtor == address(0) || creditor == address(0)) revert ZeroAddress();` in `consolidateTransfers` (the shared chokepoint, per row). ~15–22 gas/row.
- **Files:** `WERC7575ShareToken.sol` (`consolidateTransfers`).
- **Tests:** `BatchTransferZeroAddress.t.sol` (5 tests incl. a non-first row).

Code edited (before → after)

`consolidateTransfers` — reject zero-address settlement entries (`WERC7575ShareToken.sol`)

```
// AFTER
            address creditor = creditors[i];
            uint256 amount = amounts[i];

            // Reject zero-address settlement entries: a creditor of address(0) would shuffle real
            // balance to the zero address with no totalSupply change (silent value loss + Sigma-balances
            // drift). The standard transfer/transferFrom paths reject address(0); the batch paths must
            too.
            if (debtor == address(0) || creditor == address(0)) revert ZeroAddress();

            // Skip self-transfers (debtor == creditor)
            if (debtor != creditor) {
```

2.0.23 Batch round (`1ce4867`)

- **M-05 (Medium) — WERC `max*` ignored the share-token pause:** deposit/redeem route through the share token's `whenNotPaused` mint/burn, but `max*` only checked the vault's pause. All four now also check `!_shareToken.paused()`.
- **L-01 (Low) — async `maxDeposit` / `maxMint` zeroed on a deactivated vault:** they were `isActive`-gated though claiming a fulfilled deposit stays executable when inactive. Now return `claimableDepositAssets` / `claimableDepositShares` unconditionally.
- **H-01 (monitoring) — surface `totalCancelRedeemShares` in `VaultMetrics`** (per-controller cancel fields deliberately omitted to avoid extra SLOADs in paginated getters; already exposed via ERC-7887 getters).
- **Files:** `WERC7575Vault.sol`, `ERC7575VaultUpgradeable.sol`, `IVaultMetrics.sol`. **Tests:** `WERCMaxFunctionsM03.t.sol`, `MigrateAndUnregisterVault.t.sol`.

Code edited (before → after)

`maxDeposit` / `maxMint` — drop the `isActive` gate on the claimable amount (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function maxDeposit(address controller) public view virtual returns (uint256) {
        VaultStorage storage $ = _getVaultStorage();
        return $.isActive ? $.claimableDepositAssets[controller] : 0;
    }
    function maxMint(address controller) public view virtual returns (uint256) {
        VaultStorage storage $ = _getVaultStorage();
        return $.isActive ? $.claimableDepositShares[controller] : 0;
    }
```

```
// AFTER
    function maxDeposit(address controller) public view virtual returns (uint256) {
        return _getVaultStorage().claimableDepositAssets[controller]; // claim stays executable when
        inactive
    }
    function maxMint(address controller) public view virtual returns (uint256) {
        return _getVaultStorage().claimableDepositShares[controller];
```

```
    }
```

`max*` — also check the share-token pause (`WERC7575Vault.sol`)

```
// BEFORE
    function maxDeposit(address receiver) public view returns (uint256) {
        return (_isActive && !paused() && _shareToken.isKycVerified(receiver)) ? type(uint256).max : 0;
    }
    function maxWithdraw(address owner) public view returns (uint256) {
        if (paused()) return 0;
        return Math.min(_convertToAssets(_shareToken.balanceOf(owner), Math.Rounding.Floor),
        totalAssets());
    }
```

```
// AFTER
    function maxDeposit(address receiver) public view returns (uint256) {
        return (_isActive && !paused() && !_shareToken.paused() && _shareToken.isKycVerified(receiver)) ?
        type(uint256).max : 0;
    }
    function maxWithdraw(address owner) public view returns (uint256) {
        if (paused() || _shareToken.paused()) return 0;
        return Math.min(_convertToAssets(_shareToken.balanceOf(owner), Math.Rounding.Floor),
        totalAssets());
    }
```

`VaultMetrics` — expose `totalCancelRedeemShares` (`interfaces/IVaultMetrics.sol`)

```
// BEFORE
        uint256 totalCancelDepositAssets; // ERC7887 cancelation assets
```

```
// AFTER
        uint256 totalCancelDepositAssets; // ERC7887 deposit-cancelation assets (pending + claimable)
        uint256 totalCancelRedeemShares; // ERC7887 redeem-cancelation SHARES escrowed (pending +
        claimable)
```

2.0.24 Batch round (`cd543d8`)

- **M-05 (Medium) — WERC `maxWithdraw` / `maxRedeem` must reflect owner KYC and self-allowance:** redemption calls `burn(owner)` (KYC-gated) and spends the owner's validator-issued self-allowance, neither of which the prior `max*` accounted for. Now `return 0` unless `isKycVerified(owner)`, else the min of owner share value, self-allowance, and vault liquidity. (Corrected a misleading comment that claimed burning is not KYC-gated — it is, in `burn()`.)
- **L-04 (Low) — deploy role handoff:** VALIDATOR/KYC_ADMIN/REVENUE_ADMIN/INVESTMENT_MANAGER no longer silently default to the deployer (read via `_requiredEnvAddr`); WERC role handoff split into `_assignWERC7575Roles`, called **last** so the deployer keeps `kycAdmin` through privileged wiring.
- **L-05 (Doc) — stale upgrade script:** DANGER banner that the hard-coded Bepolia proxies must be REDEPLOYED, not upgraded, across the incompatible storage-layout change.
- **M-03 (Doc) — `rBalanceFlags` trust:** documented the by-design trusted-validator model.

- **Files:** `WERC7575Vault.sol`, `script/DeployBerachainMainnet.s.sol`, `script/UpgradeDeployedContracts.s.`
  `WERC7575ShareToken.sol`. **Tests:** `WERCMaxFunctionsM03.t.sol`, `DeployBerachainMainnetFork.t.sol`.

Code edited (before → after)

Roles must be explicit; WERC role handoff deferred to last (`script/DeployBerachainMainnet.s.sol`)

```
// BEFORE
        address validator = _envAddr("VALIDATOR_ADDRESS", deployer);          // silent default to
        deployer
        address kycAdmin = _envAddr("KYC_ADMIN_ADDRESS", deployer);
        address revenueAdmin = _envAddr("REVENUE_ADMIN_ADDRESS", deployer);
        address investmentManager = _envAddr("INVESTMENT_MANAGER_ADDRESS", deployer);
        // ...
        _deployWERC7575(d);
        _wireWERC7575(d, validator, kycAdmin, revenueAdmin);                   // kycAdmin moved off
deployer too early
        _deployUpgradeable(d, deployer);
        _wireUpgradeable(d, investmentManager);
```

```
// AFTER
        address validator = _requiredEnvAddr("VALIDATOR_ADDRESS");            // no default — must be set
        address kycAdmin = _requiredEnvAddr("KYC_ADMIN_ADDRESS");
        address revenueAdmin = _requiredEnvAddr("REVENUE_ADMIN_ADDRESS");
        address investmentManager = _requiredEnvAddr("INVESTMENT_MANAGER_ADDRESS");
        // ...
        _deployWERC7575(d);
        _wireWERC7575(d);
        _deployUpgradeable(d, deployer);
        _wireUpgradeable(d, investmentManager);
        // Hand off the WERC7575 roles LAST. _wireUpgradeable needs the deployer to still be the
        // kycAdmin (it calls wShare.setKycVerified(share, true)); reassigning kycAdmin earlier would
        // make that onlyKycAdmin call revert whenever KYC_ADMIN_ADDRESS != deployer.
        _assignWERC7575Roles(d, validator, kycAdmin, revenueAdmin);
```

`_requiredEnvAddr` — no-default role getter, new (`script/DeployBerachainMainnet.s.sol`)

```
// AFTER (new)
    function _requiredEnvAddr(string memory key) internal view returns (address) {
        try vm.envAddress(key) returns (address v) {
            require(v != address(0), string.concat(key, ": role address must not be zero"));
            return v;
        } catch {
            revert(string.concat(key, ": set this role explicitly before deploying (no default to
            deployer)"));
        }
    }
```

`maxWithdraw` / `maxRedeem` — gate on owner KYC + self-allowance (`WERC7575Vault.sol`)

```
// BEFORE
    function maxWithdraw(address owner) public view returns (uint256) {
        if (paused() || _shareToken.paused()) return 0;
```

```
        return Math.min(_convertToAssets(_shareToken.balanceOf(owner), Math.Rounding.Floor),
        totalAssets());
    }
    function maxRedeem(address owner) public view returns (uint256) {
        if (paused() || _shareToken.paused()) return 0;
        return Math.min(_shareToken.balanceOf(owner), _convertToShares(totalAssets(),
        Math.Rounding.Floor));
    }
```

```
// AFTER
    function maxWithdraw(address owner) public view returns (uint256) {
        if (paused() || _shareToken.paused() || !_shareToken.isKycVerified(owner)) return 0;
        uint256 byShares = _convertToAssets(_shareToken.balanceOf(owner), Math.Rounding.Floor);
        uint256 byAllowance = _convertToAssets(_shareToken.allowance(owner, owner), Math.Rounding.Floor);
        return Math.min(Math.min(byShares, byAllowance), totalAssets());
    }
    function maxRedeem(address owner) public view returns (uint256) {
        if (paused() || _shareToken.paused() || !_shareToken.isKycVerified(owner)) return 0;
        uint256 byBalance = Math.min(_shareToken.balanceOf(owner), _shareToken.allowance(owner, owner));
        return Math.min(byBalance, _convertToShares(totalAssets(), Math.Rounding.Floor));
    }
```

2.0.25 Audit batch 4 (`cf912f1`)

- **H-01 (High) — `cancelDepositRequest` dropped a controller holding a claimable deposit:** it removed the controller from `activeDepositRequesters` unconditionally, but a controller can hold a *claimable* deposit while cancelling a second *pending* one; claimable deposits were tracked only by that set, so migration could pass quiescence with a live claim. Fix: `if ($.claimableDepositShares[controller] == 0) { remove(controller); }`.
- **H-03 (High) — deploy ownership handoff:** requires `OWNER_ADDRESS`; starts a 2-step `Ownable2Step` transfer of all 8 contracts **last**, after all owner-gated wiring.
- **M-01 (Medium) — request getters ignored `requestId`:** the four ERC-7540 request getters returned this-vault state for any id; now `if (requestId != REQUEST_ID) return 0;`.
- **L-01 (Low) — WERC `maxRedeem` sub-scalingFactor dust:** dust converting to 0 assets would make `redeem` revert; `maxRedeem` now returns 0 for it.
- **L-02 (Low) — `isOperator` missing `view`** in `IERC7540Operator`.
- **H-02 (Doc) — investment target must be WERC** (the only supported target) — documented on `withdrawFromInvestment`.
- **Files:** `ERC7575VaultUpgradeable.sol`, `WERC7575Vault.sol`, `IERC7540.sol`, `script/DeployBerachainMainne`
  Tests: `MigrateAndUnregisterVault.t.sol`, `WERCMaxFunctionsM03.t.sol`, `DeployBerachainMainnetFork.t.`

Code edited (before → after)

`cancelDepositRequest` — only drop from the active set when no claimable deposit remains (H-01) (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        // New deposit requests are blocked while pendingCancelDepositAssets[controller] > 0 (set above).
        $.activeDepositRequesters.remove(controller);
        emit CancelDepositRequest(controller, REQUEST_ID, msg.sender);
```

```
// AFTER
        // Only drop the controller from the active set if it has no remaining CLAIMABLE deposit: that
        // state is tracked solely by this set (no aggregate total), so removing it while a fulfilled-
        // but-unclaimed deposit exists would let migrateBackingOut pass quiescence with a live claim.
        if ($.claimableDepositShares[controller] == 0) {
            $.activeDepositRequesters.remove(controller);
        }
        emit CancelDepositRequest(controller, REQUEST_ID, msg.sender);
```

`pendingDepositRequest` — honor **`requestId`** (M-01) (**`ERC7575VaultUpgradeable.sol`**)

```
// BEFORE
    function pendingDepositRequest(uint256, address controller) external view returns (uint256
    pendingAssets) {
        return _getVaultStorage().pendingDepositAssets[controller];   // returns this-vault state for ANY
        id
    }
```

```
// AFTER
    function pendingDepositRequest(uint256 requestId, address controller) external view returns (uint256
    pendingAssets) {
        if (requestId != REQUEST_ID) return 0; // this vault only uses requestId 0; no such request
        otherwise
        return _getVaultStorage().pendingDepositAssets[controller];
    }
```

`maxRedeem` — return 0 for sub-scalingFactor dust shares (L-01) (**`WERC7575Vault.sol`**)

```
// AFTER
        uint256 byBalance = Math.min(_shareToken.balanceOf(owner), _shareToken.allowance(owner, owner));
        uint256 shares = Math.min(byBalance, _convertToShares(totalAssets(), Math.Rounding.Floor));
        // For <18-decimal assets, a sub-scalingFactor share amount converts to 0 assets, and redeem()
        // would revert ZeroAssets. ERC-4626 forbids max* returning a revert-causing value, so report 0.
        if (_convertToAssets(shares, Math.Rounding.Floor) == 0) return 0;
        return shares;
```

`isOperator` — add **`view`** (L-02) (**`interfaces/IERC7540.sol`**)

```
// BEFORE
    function isOperator(address controller, address operator) external returns (bool status);
```

```
// AFTER
    function isOperator(address controller, address operator) external view returns (bool status);
```

Two-step ownership handoff of all 8 contracts (H-03) — new (**`script/DeployBerachainMainnet.s.sol`**)

```
// AFTER (new)
    function _transferOwnership(Deployment memory d, address newOwner) internal {
        console.log("\n-- Transferring ownership (2-step) to --", newOwner);
        d.wShare.transferOwnership(newOwner);
        d.wUsdcVault.transferOwnership(newOwner);
        d.wUsdtVault.transferOwnership(newOwner);
```

```
        d.wHoneyVault.transferOwnership(newOwner);
        d.share.transferOwnership(newOwner);
        d.usdcVault.transferOwnership(newOwner);
        d.usdtVault.transferOwnership(newOwner);
        d.honeyVault.transferOwnership(newOwner);
    }
```

---

3 Part B — Latest remediation batch (`f6b3fad` , 2026-05-25)

Resolves the most recent deep-review (SIGNOFF_2) plus standards-conformance and assurance work.

3.0.1 Decommission safety — aggregate obligation totals (completes the H-01 line)

- **Severity:** High
- **Problem:** The active-requester `EnumerableSet` s were used as the authority for decommission quiescence, but their membership desyncs across interleaved flows: `deposit` / `mint` and `redeem` / `withdraw` remove a controller on a **full** claim while a *later* request for the same controller is mid-flight, and `fulfillDeposit` moves pending→claimable without re-adding. There was **no aggregate total** for claimable deposit shares or pending redeem shares. So `migrateBackingOut` could pass quiescence while a controller still had a fulfilled-unclaimed deposit (outside the set) or an escrowed pending redeem — unregistering the vault would then brick those `onlyVaults` mint/burn claims.
- **Root cause:** Helper enumeration sets were treated as authoritative obligation trackers despite their lossy membership lifecycle.
- **Fix:** Added storage aggregates `totalClaimableDepositShares` and `totalPendingRedeemShares` , maintained at **every** mutation site so each equals the exact sum of its per-controller mapping: `+=` in `fulfillDeposit` / `fulfillDeposits` (deposit) and `requestRedeem` (redeem); `-=` in the full/partial branches of the `deposit` / `mint` claim, in `fulfillRedeem` , and in `cancelRedeemRequest` . `migrateBackingOut` now gates on these aggregates directly — reverting `CannotUnregisterVaultClaimableDeposits` / `CannotUnregisterVaultPendingRedeems` — instead of the sets (kept only as a redundant belt-and-suspenders). Both totals surfaced in `VaultMetrics` .
- **Files:** `ERC7575VaultUpgradeable.sol` , `IVaultMetrics.sol` , `IERC7575Errors.sol` .
- **Tests:** `MigrateAndUnregisterVault.t.sol::test_decommissionBlockedByClaimableDepositOutsideActiveSe` `::test_decommissionBlockedByPendingRedeem` ; plus the invariant harness below.

3.0.2 Stateful invariant harness (assurance)

- **Severity:** Informational (assurance)
- **What:** `test/ObligationInvariants.t.sol` — a `StdInvariant` harness with a try/catch-wrapped handler (extends `StdUtils` , not `Test` , so only the action functions are fuzzed) driving randomized, interleaved deposit/fulfill/claim/cancel + redeem/fulfill/claim/cancel across three controllers. Three invariants prove each migration-gating counter equals the sum of its per-controller mapping (`totalClaimableDepositShares` , `totalPendingRedeemShares` , `totalClaimableRedeemShares`). Converts the by-hand verification of those invariants into

continuous, regression-proof fuzzing.

3.0.3 Last investment-withdrawal-route guard

- **Severity:** Medium (governance-mistake)
- **Problem:** Invested backing lives as investment-share-token holdings on the share token and is redeemed back to a user-facing asset via `withdrawFromInvestment` on a registered vault with a configured investment route. Removing the **last** such route while `getInvestedAssets() > 0` would strand that backing (counted in NAV) with no on-chain redemption path until a new route is registered.
- **Root cause:** `migrateBackingOut` validated only the local vault's quiescence, not the share-token-level invested-assets / route invariant.
- **Fix:** `ShareTokenUpgradeable._migrateAndUnregisterVault` now reverts `CannotRemoveLastInvestmentWithdr` when `_calculateInvestmentAssets() > 0` **and** no remaining registered vault has a configured investment vault (`_hasInvestmentWithdrawalRoute` , bounded by `MAX_VAULTS_PER_SHARE_TOKEN`). Added a `getInvestmentVault()` getter. The owner must withdraw the invested assets before removing the last route.
- **Files:** `ShareTokenUpgradeable.sol` , `ERC7575VaultUpgradeable.sol` , `IERC7575Errors.sol` .
- **Tests:** `ERC7540ComplianceComplete.t.sol::test_M01_cannotRemoveLastInvestmentRouteWhileInvested` .

3.0.4 Investment targets are WERC-only

- **Severity:** Medium (NAV correctness, by policy)
- **Problem:** NAV values investment holdings as raw par units (`balanceOf + rBalanceOf`), which is only correct for a WERC par wrapper (18-dec, 1 share == 1 normalized asset). Supporting a generic non-WERC ERC-4626 target (non-1:1 share/asset rate) would mis-value the holding and shift value between cohorts; the WERC redemption gate (validator self-allowance) also assumes WERC.
- **Root cause:** The investment path did not constrain the target type, while the valuation and redemption logic implicitly assumed WERC.
- **Fix:** Enforce WERC-only at configuration. `ShareTokenUpgradeable.setInvestmentShareToken` probes `rBalanceOf` (via `_isWercShareToken`) and reverts `InvestmentShareTokenNotWerc` for a non-WERC token (this is the chokepoint — the token is write-once and chosen here). `ERC7575VaultUpgradeable.setInvestmentVault` re-checks on the **direct** (non-share-token) owner path. With WERC guaranteed, `_calculateInvestmentAssets` is simplified to `balanceOf + rBalanceOf` with **no `try/catch` and no branch** (`rBalanceOf` is a guaranteed mapping read), and `withdrawFromInvestment` applies the self-allowance check **unconditionally**.
- **Files:** `ShareTokenUpgradeable.sol` (`setInvestmentShareToken` , `_isWercShareToken` , `_calculateInvestmentAssets`), `ERC7575VaultUpgradeable.sol` (`setInvestmentVault` , `withdrawFromInvestment`), `IERC7575Errors.sol` .
- **Tests:** `NonWercInvestmentTarget.t.sol::test_setInvestmentShareToken_rejectsNonWerc` .

3.0.5 Investment-withdrawal NAV-dust guard

- **Severity:** Low
- **Problem:** When `withdrawFromInvestment` clamps the redeemed share amount to the share-token's investment-share balance (which need not be a clean multiple of the target's scaling,

e.g. after rBalance/batch movement), redeeming the raw amount burns sub-unit shares — dropping NAV — for assets that floor away, silently socializing that dust to holders.
- **Fix:** Snap the redeemed amount **down** to the exact share amount backing the realizable (Floor-rounded) assets, via the target's own `previewRedeem` / `convertToShares`. Only ever lowers the amount (never withdraws more than intended/available).
- **Files:** `ERC7575VaultUpgradeable.sol` (`withdrawFromInvestment`).

3.0.6 Redeem dust guard + fulfill zero-asset guard

- **Severity:** Low
- **Problem:** `fulfillRedeem` and a partial `redeem(shares)` could Floor-round to **zero assets** for a sub-`scalingFactor` share amount on a low-decimal asset, burning the escrowed shares for no payout — a dust loss socialized to remaining holders.
- **Fix:** `fulfillRedeem` rejects `assets == 0`; the partial `redeem` path reverts `ZeroAssets` likewise. The full position remains claimable (verified — no lockup): `redeem(availableShares)` always yields `assets == availableAssets > 0`, and `withdraw()` Ceil-rounds shares so a positive asset claim never burns zero shares.
- **Files:** `ERC7575VaultUpgradeable.sol` (`fulfillRedeem`, `redeem`).
- **Tests:** `FulfillRedeemZeroAssets.t.sol` (sub-scaling fulfill reverts; partial-redeem dust reverts; whole-unit claims succeed).

3.0.7 rBalance increment overflow hardening

- **Severity:** Low (defense-in-depth)
- **Problem:** rBalance increments (`rBatchTransfers` debtor disbursement, `adjustrBalance` profit, `cancelrBalanceAdjustment` loss-reversal) were inside `unchecked` blocks, so a cumulative increment could in principle wrap to a small value (corrupting the receivable component of NAV). (Wrapping requires economically-impossible cumulative magnitudes, hence Low — but cheap to harden.)
- **Fix:** Removed `unchecked` from every `_rBalances += delta` increment so they revert on overflow. The guarded subtractions/decrements stay `unchecked` (each is dominated by its branch condition or a preceding `< revert` guard) — and were tightened so only the genuinely-overflowable additions are checked.
- **Files:** `WERC7575ShareToken.sol` (`rBatchTransfers`, `adjustrBalance`, `cancelrBalanceAdjustment`).
- **Tests:** `RBalanceOverflowGuard.t.sol` (near-`type(uint256).max` increment reverts instead of wrapping).

3.0.8 `transferFrom` self-allowance correctness

- **Severity:** Low
- **Problem:** `transferFrom(from == msg.sender)` spent the owner's validator-issued self-allowance **twice** (once explicitly, once via `super.transferFrom`'s `allowance[from][msg.sender]` which is the same slot when `msg.sender == from`) — over-charging the permit for a single self-movement.
- **Fix:** Branch on `msg.sender == from`: spend the self-allowance **once**, then `_transfer` directly (routing through the overridden `_update` / custom `_balances`), matching `transfer()`. The delegated path (`msg.sender != from`) keeps the dual gate (self-allowance + caller allowance) and the recipient-KYC check.

- **Files:** `WERC7575ShareToken.sol` (`transferFrom`).
- **Tests:** `WERCMaxFunctionsM03.t.sol::test_L03_transferFromSelf_spendsSelfAllowanceOnce`, `::test_L03_delegatedTransferFrom_stillDualGated`.

3.0.9 Vault solvency visibility

- **Severity:** Low
- **Problem:** `totalAssets()` saturates to 0 when reserved obligations exceed the gross balance, hiding an asset shortfall from the public getter.
- **Fix:** `getVaultMetrics` now exposes `grossAssetBalance` and `reservedAssets` (the shortfall is derivable as `reserved > gross ? reserved - gross : 0`), mirroring exactly the reserve set `totalAssets()` subtracts.
- **Files:** `ERC7575VaultUpgradeable.sol` (`getVaultMetrics`), `IVaultMetrics.sol`.
- **Tests:** `VaultMetricsShortfall.t.sol`.

3.0.10 ERC-7540 / ERC-7887 conformance

- **Severity:** Low (standards conformance)
- **`setOperator`:** no longer reverts `operator == msg.sender`. ERC-7540 requires it to set status, emit `OperatorSet`, and return `true` for any operator; self is a harmless no-op (every auth path short-circuits on `controller == msg.sender` before consulting the operator map). The now-dead `CannotSetSelfAsOperator` error was removed.
- **2-arg `deposit` / `mint`:** the inherited ERC-4626 entrypoints now delegate with `msg.sender` as the controller (was `receiver`), per ERC-7540 — so the 2-arg form claims the caller's own fulfilled request; operators use the 3-arg form.
- **Parameter names:** `IERC7540Redeem.pendingRedeemRequest` / `claimableRedeemRequest` use `controller`; `IERC7887RedeemCancelation.claimCancelRedeemRequest` uses `(receiver, controller)`; the implementation's `claimCancelRedeemRequest` renamed its `owner` parameter to `receiver`. ABI selectors are unchanged (parameter names don't affect selectors).
- **Docblock event annotations** corrected to match the actual `Deposit` / `Withdraw` emit argument order.
- **Files:** `ERC7575VaultUpgradeable.sol`, `IERC7540.sol`, `IERC7887.sol`, `IERC7575Errors.sol`.
- **Tests:** `ERC7540AsyncEdgeCases.t.sol`, `EdgeCases_AuthorizationPaths.t.sol`, `ERC7575Security.t.sol`, `ERC7540ComplianceComplete.t.sol` updated to the conformant behavior.

3.0.11 Canonical ERC-7201 namespaced storage slots

- **Severity:** Hardening
- **Problem:** The two upgradeable contracts derived their namespaced storage base with plain `keccak256("id")` rather than the canonical ERC-7201 formula, so the slot did not reserve the standard 256-slot gap (last byte not zeroed).
- **Fix:** Use the canonical `keccak256(abi.encode(uint256(keccak256(id)) - 1)) & ~0xff` as pre-computed literals (the formula isn't a `constant` expression). The trailing zero byte reserves the gap so the struct can't collide with mapping/array data laid out from the base.
- **Files:** `ERC7575VaultUpgradeable.sol`, `ShareTokenUpgradeable.sol`.
- **Tests:** `Erc7201StorageSlots.t.sol` (asserts the canonical derivation **and** that the deployed

proxy anchors its struct at the slot — `asset` for the vault, the `assetToVault` map length for the share token). Production storage-layout compatibility is out of scope (no prior production deployment).

3.0.12 Pause-semantics documentation

- **Severity:** Doc
- **Fix:** Corrected the misleading "Contract must not be paused" NatSpec on `batchTransfers` / `rBatchTransfers` — validator settlement is intentionally **not** pause-gated (pause freezes only user-facing flows); aligned with `DESIGN_ASSUMPTIONS.md`.
- **Files:** `WERC7575ShareToken.sol`.

Code edited (before → after)

`VaultStorage` — add aggregate obligation totals (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        uint256 totalPendingDepositAssets;
        uint256 totalClaimableRedeemAssets; // Assets reserved for users who can claim them
        uint256 totalClaimableRedeemShares; // Shares held by vault that will be burned on
        redeem/withdraw
```

```
// AFTER
        uint256 totalPendingDepositAssets;
        uint256 totalClaimableRedeemAssets; // Assets reserved for users who can claim them
        uint256 totalClaimableRedeemShares; // Shares held by vault that will be burned on
        redeem/withdraw
        // Aggregate obligation totals that back the migrateBackingOut quiescence gate directly (so it
        // does not rely on the activeDeposit/RedeemRequesters sets, whose membership can desync across
        // interleaved fulfill/claim/cancel sequences):
        uint256 totalClaimableDepositShares; // Shares minted-to-vault for fulfilled-but-unclaimed
        deposits
        uint256 totalPendingRedeemShares; // Shares escrowed in the vault for pending (unfulfilled)
        redeems
```

`migrateBackingOut` — gate on the aggregate obligation totals (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        if ($.totalPendingDepositAssets != 0) revert CannotUnregisterVaultPendingDeposits();
        if ($.totalClaimableRedeemAssets != 0) revert CannotUnregisterVaultClaimableRedemptions();
```

```
// AFTER
        // Authoritative obligation gate: aggregate totals cover every in-flight request state directly,
        // so quiescence does not depend on the activeDeposit/RedeemRequesters sets (whose membership can
        // desync across interleaved fulfill/claim/cancel). Set-length checks kept as
        belt-and-suspenders.
        if ($.totalPendingDepositAssets != 0) revert CannotUnregisterVaultPendingDeposits();
        if ($.totalClaimableDepositShares != 0) revert CannotUnregisterVaultClaimableDeposits();
        if ($.totalPendingRedeemShares != 0) revert CannotUnregisterVaultPendingRedeems();
        if ($.totalClaimableRedeemAssets != 0) revert CannotUnregisterVaultClaimableRedemptions();
```

`fulfillRedeem` — reject zero-asset (dust) claims (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        assets = _convertToAssets(shares, Math.Rounding.Floor);
        if (assets > totalAssets()) {
            revert ERC20InsufficientBalance(address(this), totalAssets(), assets);
        }
```

```
// AFTER
        assets = _convertToAssets(shares, Math.Rounding.Floor);
        // Reject fulfilling a redeem that Floor-rounds to zero assets (a sub-scalingFactor `shares`
        // amount for a low-decimal asset). Without this, the claim would burn the escrowed shares for
        // zero payout — a dust loss to the redeemer, socialized to remaining holders.
        if (assets == 0) revert ZeroAssets();
        if (assets > totalAssets()) {
            revert ERC20InsufficientBalance(address(this), totalAssets(), assets);
        }
```

`_calculateInvestmentAssets` — WERC-only NAV; drop the try/catch and the branch (`ShareTokenUpgradeable.sol`)

```
// BEFORE
        totalInvestmentAssets = IERC20(investmentShareToken).balanceOf(address(this));
        try IWERC7575ShareToken(investmentShareToken).rBalanceOf(address(this)) returns (uint256 rShares)
        {
            totalInvestmentAssets += rShares;
        } catch {
            // rBalanceOf unsupported or reverted — continue with regular balance only.
        }
```

```
// AFTER
        // WERC is enforced at configuration, so rBalanceOf is a guaranteed mapping read — no try/catch.
        return IERC20(investmentShareToken).balanceOf(address(this)) +
        IWERC7575ShareToken(investmentShareToken).rBalanceOf(address(this));
```

`deposit` / `mint` (2-arg) — controller = `msg.sender`, not `receiver` (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function deposit(uint256 assets, address receiver) public virtual returns (uint256) {
        return deposit(assets, receiver, receiver);
    }
    function mint(uint256 shares, address receiver) public virtual returns (uint256) {
        return mint(shares, receiver, receiver);
    }
```

```
// AFTER
    function deposit(uint256 assets, address receiver) public virtual returns (uint256) {
        return deposit(assets, receiver, msg.sender);   // ERC-7540: 2-arg claims the CALLER's request
    }
    function mint(uint256 shares, address receiver) public virtual returns (uint256) {
        return mint(shares, receiver, msg.sender);
    }
```

`transferFrom` — spend the self-allowance only once on a self-movement (L-03) (`WERC7575ShareToken.sol`)

```
// BEFORE
    function transferFrom(address from, address to, uint256 value) public override whenNotPaused returns
    (bool) {
        if (!isKycVerified[to]) revert KycRequired();
        _spendAllowance(from, from, value);
        return super.transferFrom(from, to, value);   // spends allowance[from][msg.sender] ==
        self-allowance AGAIN
    }
```

```
// AFTER
    function transferFrom(address from, address to, uint256 value) public override whenNotPaused returns
    (bool) {
        if (!isKycVerified[to]) revert KycRequired();
        _spendAllowance(from, from, value); // self-allowance gate (validator-issued restricted-token
proof)
        if (msg.sender == from) {
            // Self-movement: do NOT consume the self-allowance a second time via the delegated path.
            _transfer(from, to, value);
            return true;
        }
        return super.transferFrom(from, to, value); // additionally spends the delegated
        allowance[from][msg.sender]
    }
```

`setOperator` — allow self-operator per ERC-7540 (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function setOperator(address operator, bool approved) public virtual returns (bool) {
        if (operator == msg.sender) revert CannotSetSelfAsOperator();
        // ...
```

```
// AFTER
    function setOperator(address operator, bool approved) public virtual returns (bool) {
        // ERC-7540 mandates this MUST set status, emit, and return true for any operator — including
        // operator == msg.sender. Self-operator is a harmless no-op (auth paths short-circuit on
        // controller == msg.sender before consulting operators[controller][operator]).
        // ...
```

---

4 Part C — Final hardening + cleanup batch (`7e40f39` , `7e346f5` , 2026-05-26/27)

This batch followed three further deep-review passes. The first commit (`7e40f39`) bundles the security/correctness hardening plus the accumulated gas/cleanup and deployment artifacts; the second (`7e346f5`) clears the remaining low-severity items from the final pass. The closing review reported **no open Critical/High/Medium/Low contract-code findings**.

4.1 `7e40f39` — investment/migration hardening, ERC-20 getter removal, gas/cleanup, deploy docs

4.1.1 Investment withdrawal rewritten to asset-denominated `withdraw` clamped by `maxWithdraw` (supersedes the Part B dust guard)

- **Severity:** Medium (NAV correctness) / cleanup
- **Problem:** `withdrawFromInvestment(amount)` previously converted the requested assets to shares (`previewWithdraw`), hand-clamped them to the share token's balance, then "snapped" the share count down to a clean multiple to avoid burning sub-unit shares for assets that floor away (the Part B NAV-dust guard), and finally `redeem` ed and pre-checked the validator self-allowance. That was a large, manual reconstruction of accounting the target vault already exposes, and the share-denominated `redeem` was what created the dust hazard in the first place.
- **Fix:** The function now reads `cap = maxWithdraw(shareToken)` and calls the target's **asset-denominated** `withdraw(min(amount, cap), thisVault, shareToken)` . `maxWithdraw` already folds in the three binding limits — the share token's WERC balance (floored to whole asset units), the validator self-allowance (the WERC redemption gate), and the target's liquid assets — and for the par wrapper `convertToShares(assets) == assets * scalingFactor` exactly, so `withdraw` burns a **clean multiple** and no sub-unit NAV dust can be socialized. `type(uint256).max` still means "withdraw everything available." The manual self-allowance pre-check is gone (the clamp covers partial cases; a full freeze collapses the cap to 0 and the target's `withdraw` reverts with its own error), and the now-orphaned `InvestmentSelfAllowanceMissing` error was removed. The dust-snap block, `previewWithdraw` / `previewRedeem` / `convertToShares` dance, and per-call allowance read were all deleted.
- **Files:** `ERC7575VaultUpgradeable.sol` (`withdrawFromInvestment`), `IERC7575Errors.sol` (removed `InvestmentSelfAllowanceMissing`).
- **Tests:** existing investment-flow + `type(uint256).max` withdraw-all coverage (`ERC7540ComplianceComplete` , `CompleteFlowWalkthrough`).

4.1.2 `setInvestmentVault` restricted to the share-token-mediated path (strengthens `4474f87`)

- **Severity:** Medium
- **Problem:** The operator allowance `allowance(shareToken, vault)` on the investment share token — which `withdrawFromInvestment` 's `withdraw` consumes — is granted **only** by `ShareTokenUpgradeable._configureVaultInvestmentSettings` (set to `max` at route configuration). The legacy **direct owner** path of `setInvestmentVault` set `$.investmentVault` but could not grant that allowance. A route configured that way could report a positive `maxWithdraw` yet **revert at withdrawal** on the missing operator allowance, with no clean recovery (the investment-share-token setter is one-shot and the vault is already registered).
- **Fix:** Removed the owner branch from `setInvestmentVault` 's authorization; it is now callable **only by the share token** (`msg.sender == shareToken` , else `Unauthorized`), so route setup and the allowance grant are always coupled through `_configureVaultInvestmentSettings` . The now-dead WERC-probe defense-in-depth block (and its orphaned `IWERC7575ShareToken` import) were dropped.
- **Files:** `ERC7575VaultUpgradeable.sol` (`setInvestmentVault`).
- **Tests:** `InvestmentVaultShareCheck.t.sol::test_setInvestmentVault_directOwnerCall_reverts` ,

`ERC7540ComplianceComplete.t.sol::test_InvestmentVault_DirectOwnerCall_Reverts`.

4.1.3 Migration destination-solvency postcondition (`MigrationNavNotPreserved`)

- **Severity:** Medium (hardening of an otherwise accepted par residual)
- **Problem:** `_migrateAndUnregisterVault` drains the source vault and transfers the computed compensating asset into the destination, asserting NAV preservation in its NatSpec. But `totalAssets()` is net of reserves and saturates to zero, so if the destination is **already under-reserved** (e.g. after an external asset loss/depeg), the injected compensation first fills that hidden deficit and destination `totalAssets()` rises by **less** than the normalized backing removed from the source — a silent NAV reduction. (Under the contract's own invariants `balance >= reservedAssets` always holds, so the precondition is only reachable via external loss — the accepted par/no-oracle residual — but the NatSpec claimed preservation unconditionally.)
- **Fix:** After the compensation transfer, the migration now snapshots the destination's `totalAssets()` before/after and requires the **real normalized delta** to cover the removed backing: `(toAssetsAfter - toAssetsBefore) * toScaling >= moved * fromScaling`, reverting the new `MigrationNavNotPreserved` otherwise. Solvent migrations always pass (delta equals the injected amount, and `amountIn * toScaling >= moved * fromScaling` by the existing ceil rounding); an under-reserved destination reverts loudly and atomically instead of silently losing NAV.
- **Files:** `ShareTokenUpgradeable.sol` (`_migrateAndUnregisterVault`), `IERC7575Errors.sol` (`MigrationNavNotPreserved`).
- **Tests:** `MigrateAndUnregisterVault.t.sol::test_revertIfDestinationUnderReserved` (simulates an external loss, asserts revert + atomic rollback); the 6→18 / 18→6 cross-decimal solvent-migration tests confirm no false positive.

4.1.4 Remove the ERC-20-like `totalSupply()` / `balanceOf()` from the async vault

- **Severity:** Low (interface hygiene)
- **Problem:** `ERC7575VaultUpgradeable` exposed `totalSupply()` and `balanceOf(address)` that delegated to the share token. They were in no interface, used by no production caller, and inconsistent with `WERC7575Vault` (which has neither). Worse, an ERC-7575 vault is **not** the share token: these returned the share token's **system-wide** values, so an integrator treating the vault as the shares would read wrong, cross-vault numbers.
- **Fix:** Removed both functions; integrations read `share()` and call ERC-20 there (matching `WERC7575Vault` and the ERC-7575 vault/share separation). `supportsInterface` is unaffected. Repointed the one test and three debug scripts at the share token.
- **Files:** `ERC7575VaultUpgradeable.sol`; `ERC7540ComplianceComplete.t.sol`; `script/{DebugDeposit,Interac`

4.1.5 Full-share redeem-claim predicate + cross-decimal migration tests

- **Severity:** Low / assurance
- **Fix:** The async `redeem` full-claim branch keys on `shares == availableShares` (assigning `assets = availableAssets` directly, clearing both claimable slots), with the zero-asset dust guard scoped to the partial branch — removing residual claimable dust on a full claim. Added 6→18 and 18→6 cross-decimal migration tests asserting `amountIn = ceil(moved * fromScaling / toScaling)` and normalized-backing preservation.

- **Files:** `ERC7575VaultUpgradeable.sol` (`redeem`), `MigrateAndUnregisterVault.t.sol` .

4.1.6 WERC gas/cleanup

- **Severity:** Hardening (gas; behavior unchanged)
- **Fix:** Made `WERC7575Vault` 's constructor-only fields `immutable` (non-upgradeable contract → safe, fewer hot-path SLOADs); consolidated duplicate WERC errors onto the shared `IERC7575Errors` set; tidied `unchecked` blocks (balance ops only — **rBalance increments stay checked**, preserving the overflow revert from Part B); and packed `rBatchTransfers` ' debtor/creditor flags into a single `uint256` bitmap (`MAX_BATCH_SIZE` kept <= 128 so the account count stays <= 256 bits).
- **Files:** `WERC7575Vault.sol` , `WERC7575ShareToken.sol` .

4.1.7 Deployment artifacts + NatSpec corrections

- **Severity:** Docs / deployment
- **Fix:** Added the committed Berachain mainnet runbook (`BERACHAIN_DEPLOYMENT.md`) and env template (`.env.mainnet.example` , no secrets). Corrected the `pause()` NatSpec (it gates user `transfer` / `mint` / `burn` , **not** batch settlement or rBalance adjustments), the `batchTransfers` NatSpec (nets `_balances` only — rBalance movement lives in `rBatchTransfers`), the per-vault operator NatSpec, and the `IERC7540` event-name comments (`DepositRequest` / `RedeemRequest`).
- **Files:** `BERACHAIN_DEPLOYMENT.md` , `.env.mainnet.example` , `WERC7575ShareToken.sol` , `ERC7575VaultUpgradeable.sol` , `IERC7540.sol` .

Code edited (before → after)

`withdrawFromInvestment` — asset-denominated `withdraw` clamped by `maxWithdraw` (supersedes the Part B dust-snap) (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
        uint256 maxShares = investmentShareToken.balanceOf(shareToken_);
        uint256 shares = IERC7575($.investmentVault).previewWithdraw(amount);
        uint256 minShares = shares < maxShares ? shares : maxShares;
        if (minShares == 0) revert ZeroSharesCalculated();

        uint256 realizableAssets = IERC7575($.investmentVault).previewRedeem(minShares);
        if (realizableAssets == 0) revert ZeroAssetsCalculated();
        uint256 cleanShares = IERC7575($.investmentVault).convertToShares(realizableAssets);  // manual
        dust-snap
        if (cleanShares < minShares) minShares = cleanShares;

        uint256 current = investmentShareToken.allowance(shareToken_, shareToken_);
        if (current < minShares) revert InvestmentSelfAllowanceMissing(minShares, current);

        IERC7575($.investmentVault).redeem(minShares, address(this), shareToken_);   // share-denominated
=> dust hazard
```

```
// AFTER
        uint256 cap = IERC7575($.investmentVault).maxWithdraw(shareToken_);
        uint256 assets = amount < cap ? amount : cap;
```

```
        // ShareToken is owner, this vault is receiver. The maxWithdraw clamp already folds in the WERC
        // balance, the validator self-allowance, and the target's liquidity — and for the par wrapper
        // convertToShares(assets) == assets * scalingFactor exactly, so withdraw() burns a clean
        multiple
        // (no sub-unit NAV dust). Remaining reverts come from withdraw() itself, surfaced as its own
        error.
        IERC7575($.investmentVault).withdraw(assets, address(this), shareToken_);
```

`setInvestmentVault` — share-token-mediated only (drop the direct owner path) (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function setInvestmentVault(IERC7575 investmentVault_) external {
        VaultStorage storage $ = _getVaultStorage();
        if (msg.sender != owner() && msg.sender != $.shareToken) revert Unauthorized();  // owner path
        can't grant allowance
        if (address(investmentVault_) == address(0)) revert InvalidVault();
```

```
// AFTER
    function setInvestmentVault(IERC7575 investmentVault_) external {
        VaultStorage storage $ = _getVaultStorage();
        if (msg.sender != $.shareToken) revert Unauthorized();   // route setup + allowance grant stay
        coupled
        if (address(investmentVault_) == address(0)) revert InvalidVault();
```

`_migrateAndUnregisterVault` — destination-solvency postcondition (`ShareTokenUpgradeable.sol`)

```
// BEFORE
            uint256 amountIn = Math.mulDiv(moved, ERC7575VaultUpgradeable(fromVault).getScalingFactor(),
            ERC7575VaultUpgradeable(toVault).getScalingFactor(), Math.Rounding.Ceil);
            if (amountIn > 0) {
                SafeTokenTransfers.safeTransferFrom(toAsset, msg.sender, toVault, amountIn);
            }
            emit VaultBackingMigrated(fromAsset, fromVault, moved, toAsset, toVault, amountIn);
```

```
// AFTER
            uint256 fromScaling = ERC7575VaultUpgradeable(fromVault).getScalingFactor();
            uint256 toScaling = ERC7575VaultUpgradeable(toVault).getScalingFactor();
            uint256 amountIn = Math.mulDiv(moved, fromScaling, toScaling, Math.Rounding.Ceil);

            uint256 toAssetsBefore = ERC7575VaultUpgradeable(toVault).totalAssets();
            if (amountIn > 0) {
                SafeTokenTransfers.safeTransferFrom(toAsset, msg.sender, toVault, amountIn);
            }
            // Require the REAL normalized delta to cover the removed backing. totalAssets() is net of
            // reserves and saturates to 0, so an already-under-reserved destination would otherwise
            // absorb the compensation into its hidden deficit — a silent NAV reduction.
            uint256 normalizedDelta = (ERC7575VaultUpgradeable(toVault).totalAssets() - toAssetsBefore) *
            toScaling;
            if (normalizedDelta < moved * fromScaling) revert MigrationNavNotPreserved();

            emit VaultBackingMigrated(fromAsset, fromVault, moved, toAsset, toVault, amountIn);
```

Remove the ERC-20-like `totalSupply()` / `balanceOf()` from the async vault

(`ERC7575VaultUpgradeable.sol`)

```
// BEFORE
    function totalSupply() public view virtual returns (uint256) {
        // Returns the share token's SYSTEM-WIDE supply — wrong, cross-vault number for an integrator
        return IERC20Metadata(_getVaultStorage().shareToken).totalSupply();
    }
    function balanceOf(address account) public view virtual returns (uint256) {
        return IERC20Metadata(_getVaultStorage().shareToken).balanceOf(account);
    }
```

```
// AFTER
    // Removed. An ERC-7575 vault is NOT the share token; integrations read share() and call ERC-20
    // there (matching WERC7575Vault and the vault/share separation). supportsInterface is unaffected.
```

Make `WERC7575Vault` constructor-only fields `immutable` (`WERC7575Vault.sol`)

```
// BEFORE
    address private _asset;
    uint64 private _scalingFactor;
    bool private _isActive;
    WERC7575ShareToken private _shareToken;
```

```
// AFTER
    // Set once in the constructor, read on the hot conversion/transfer paths — immutable (bytecode
    // reads, no SLOAD). Safe because this contract is non-upgradeable (unlike the proxy-backed vault).
    address private immutable _asset;
    uint64 private immutable _scalingFactor;
    WERC7575ShareToken private immutable _shareToken;
    bool private _isActive; // storage — mutated by setVaultActive()
```

4.2 `7e346f5` — final-review low items + doc/ABI hygiene

4.2.1 Reject `account == address(0)` in revenue rBalance adjustment / cancellation

- **Severity:** Low
- **Problem:** `adjustrBalance` and `cancelrBalanceAdjustment` (revenue-admin) did not reject a zero `account`, allowing meaningless zero-address rBalance ledger entries/events (ledger pollution; no fund-loss).
- **Fix:** Added `if (account == address(0)) revert ZeroAddress();` to both, mirroring the batch-settlement zero-address rejection.
- **Files:** `WERC7575ShareToken.sol`.
- **Tests:** `RBalanceOverflowGuard.t.sol` (`test_L01_adjustrBalanceRejectsZeroAddress`, `test_L01_cancelrBalanceAdjustmentRejectsZeroAddress`).

4.2.2 Drop the duplicate `VaultUpdate` event from the vault-side `IERC7575`

- **Severity:** Low (ABI/docs hygiene)
- **Fix:** ERC-7575 places `VaultUpdate` on the share-to-vault lookup (share) side. It was also declared on the vault-side `IERC7575`, which the vault never emits; removed it there (kept in

`IERC7575Share`). Events don't contribute to `interfaceId`, so `type(IERC7575).interfaceId` stays `0x2f0a18c5` — confirmed by the interface-ID tests.

- **Files:** `interfaces/IERC7575.sol`.

4.2.3 Documentation corrections (no behavior change)

- **Severity:** Docs
- **Fix:** The `rBatchTransfers` architecture block claimed a `nonReentrant` modifier it does not have — corrected to the real basis (no external calls → no reentry vector, CEI). Five vault NatSpec lines that said operators are approved "on the share token" now say "on this vault" (operators are per-vault), and the share-token header/feature/security comments were aligned. Stale `getTotalNormalizedAssets` references were renamed to `getCirculatingSupplyAndAssets`.
- **Files:** `WERC7575ShareToken.sol`, `ERC7575VaultUpgradeable.sol`, `ShareTokenUpgradeable.sol`, tests.

4.2.4 Correctness re-verification (no change)

- Re-derived that `getClaimableSharesAndNormalizedAssets` correctly returns only `totalClaimableRedeemShare` a share bucket is removed from circulating supply **iff** its assets are excluded from `totalAssets()`. Only the claimable-redeem bucket reserves assets; cancel-redeem returns shares (reserves no assets), so pending/claimable cancel-redeem shares correctly remain counted. No code change.

Code edited (before → after)

`adjustrBalance` / `cancelrBalanceAdjustment` — reject zero-address account (L-01) (`WERC7575ShareToken.sol`)

```
// BEFORE
    function adjustrBalance(address account, uint256 ts, uint256 amounti, uint256 amountr) external
    onlyRevenueAdmin {
        if (_rBalanceAdjustments[account][ts][0] != 0) revert RBalanceAdjustmentAlreadyApplied();
```

```
// AFTER
    function adjustrBalance(address account, uint256 ts, uint256 amounti, uint256 amountr) external
    onlyRevenueAdmin {
        if (account == address(0)) revert ZeroAddress();
        if (_rBalanceAdjustments[account][ts][0] != 0) revert RBalanceAdjustmentAlreadyApplied();
```

`IERC7575` — drop the duplicate `VaultUpdate` event (vault side) (L-04) (`interfaces/IERC7575.sol`)

```
// BEFORE
interface IERC7575 {
    /// @dev Emitted when a vault address is updated for a specific asset.
    event VaultUpdate(address indexed asset, address vault);   // belongs on the share side only

    /// @dev Returns the address of the share token.
    function share() external view returns (address);
```

```
// AFTER
interface IERC7575 {
    /// @dev Returns the address of the share token.
    function share() external view returns (address);
```

`rBatchTransfers` — correct the reentrancy NatSpec (no `nonReentrant`) (L-02) (`WERC7575ShareToken.sol`)

```
// AFTER (doc)
     * 4. REENTRANCY PROTECTION
     *    - No nonReentrant modifier: the function makes no external calls, so reentrancy is impossible
     *    - All state changes precede the Transfer events (CEI pattern); no callbacks to untrusted code
     *    - (Add nonReentrant if a future change introduces an external call)
```

Operators are per-vault, not "on the share token" (L-03) (`ERC7575VaultUpgradeable.sol`)

```
// BEFORE (doc)
     * Operator must be approved via setOperator() on the share token
```

```
// AFTER (doc)
     * Operator must be approved via setOperator() on this vault
```

---

5 Accepted residual risks (explicit, not defects)

Intentional trust properties, documented in `docs/DESIGN_ASSUMPTIONS.md` and `docs/RBALANCE_MODEL.md` (§4b owner sign-off):

- **Par accounting** across registered stablecoins (no on-chain oracle / depeg circuit breaker); protection is the operator gate + depeg runbook (C-01).
- **rBalance is a trusted receivable mark.** Validator (`rBatchTransfers`) and revenue-admin (`adjustrBalance`) are trusted roles that affect NAV; migrate these EOAs to multisig before TVL grows.
- **Batch settlement KYC** is enforced off-chain by the validator; on-chain batch paths skip KYC.
- **KYC is recipient-gating**, not a freeze. A genuine freeze is achievable by the validator revoking an account's self-allowance (a zero-value self-permit blocks all sends).
- **Pause** freezes only user-facing flows, not privileged settlement/accounting.
- **Mixed-role batch netting** (R-6): a flagged account must carry one economic role per batch; the net-creditor floor is irreversible — off-chain batch-construction discipline.
- **rBalance adjustment cancellation** can become non-executable after later settlement; **events** are not replay-complete (off-chain reconciliation required). Hardening declined by the owner.

---

6 Verification

- `forge build` — clean (only pre-existing test/script lint warnings).
- `forge test` — **804 passed, 0 failed, 0 skipped** (100 suites).

- Canonical ERC-165 interface IDs verified against the live contract: `IERC7575 0x2f0a18c5`, `IERC7575Share 0xf815c03d`, `IERC7540Operator 0xe3bc4e65`, `IERC7540Deposit 0xce3bbe50`, `IERC7540Redeem 0x620ee8e4`, `IERC7887DepositCancelation 0x8bf840e3`, `IERC7887RedeemCancelation 0xe7`
- Regression suites added across the range: `MigrateAndUnregisterVault`, `WERCMigrateAndUnregisterVault`, `WERCMaxFunctionsM03`, `ZeroAddressValidationM08`, `BatchTransferZeroAddress`, `PermitERC1271`, `InvestmentVaultShareCheck`, `OperatorInterfaceCompliance`, `AuditUpgradeFlow`, `ObligationInvariants`, `Erc7201StorageSlots`, `RBalanceOverflowGuard`, `VaultMetricsShortfall`, `FulfillRedeemZeroAssets`, `NonWercInvestmentTarget`.